ES32029
Advanced Econometrics and Data Analytics

Semester 2 AY 2025-26

Coursework

Word Count: 7,754

April 2026

Group 34

Table of Contents

ES32039 Coursework – Part 1 3

1. Model selection 3

2. Tree and ensemble methods 18

ES32039 coursework – part 2 33

1. Introduction 33

2. Data preparation 33

3. Methodology and results 35

4. Robustness checks 49

5. Discussion 55

6. Conclusion 57

7. References 58

ES32039 Coursework – Part 1

1. Model selection

(a) Ordinary Least Squares (OLS) chooses parameters by minimising Residual Sum of Squares (RSS). Under standard linear assumptions, this estimator is unbiased, but prone to overfitting, especially when explanatory variables are highly correlated. Overfitting can lead to large coefficients and high out-of-sample Mean Square Error (MSE). Ridge and Lasso counteract this by adding a penalty to the loss function, which shrinks coefficients and aims to reduce MSE.

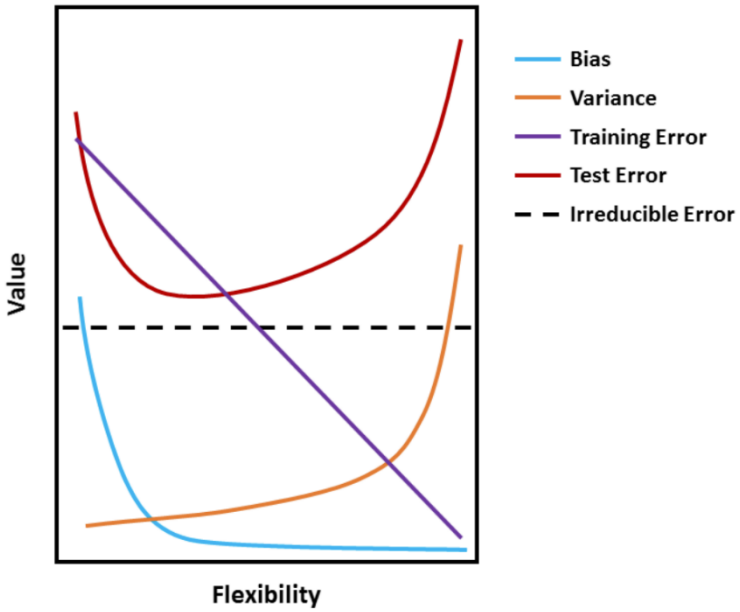
Suppose we want to predict salary of football players using goals and shots on target:

$$Salary_i = \beta_0 + \beta_1 goals_i + \beta_2 shots_i$$

These explanatory variables will be highly correlated. Therefore, OLS will struggle to determine their individual effects. As OLS is a flexible model, this can lead to large, unstable coefficients with opposing signs to fit a regression line through the training data. This negatively impacts the model's ability to reliably predict on unseen data.

Out-of-sample prediction error can be decomposed into bias^2 + variance. If we wish to reduce MSE relative to OLS, we must find a way to reduce variance, even if this comes at the cost of introducing bias. *Figure 1* shows this trade-off. Bias is decreasing in model flexibility, whilst variance is increasing. OLS is to the right of this diagram, producing a large test error. Ridge and Lasso are alternative estimators, which penalise large coefficients. This reduces model flexibility and variance, often improving test error, despite introducing bias.

Figure 1: The Bias-Variance Trade-Off



Ridge penalises coefficients by adding an L2 penalty to its loss function, which is proportional to the sum of squared coefficients. The severity of the penalty is given by λ.

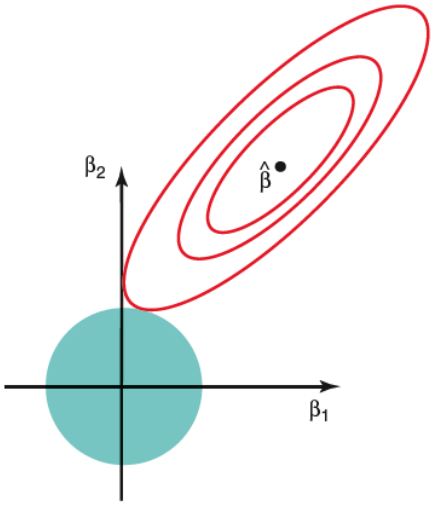
$$\min_{\beta} \left(RSS + \lambda \sum_{i=1}^{p} \beta_i^2 \right)$$

Using our previous example, the equivalent constrained minimisation problem is:

$$\begin{aligned} &\min_{\beta_1, \beta_2} \quad RSS + \lambda(\beta_1^2 + \beta_2^2) \\ &= \min_{\beta} \quad RSS \\ &\quad s.t. \quad \beta_1^2 + \beta_2^2 \leq s \end{aligned}$$

where the magnitude of s reflects the lenience of the penalty. As we can see, the constraint is circular:

Figure 2: Ridge graphical representation



$\hat{\beta}$ shows where RSS is minimised and the red ellipses show RSS contours. Ridge finds the point with smallest RSS that still satisfies the constraint, which is the tangent between the ellipse and constraint. The circular shape reflects the diminishing marginal benefit of shrinking a coefficient, which explains why Ridge will not set parameters to zero, but can set them very small.

Instead, Lasso's loss function introduces an L1 penalty, which is proportional to the sum of absolute value of coefficients. Similarly, λ determines the strength of the penalty.

$$\min_{\beta} \left(RSS + \lambda \sum_{i=1}^{p} |\beta_i| \right)$$

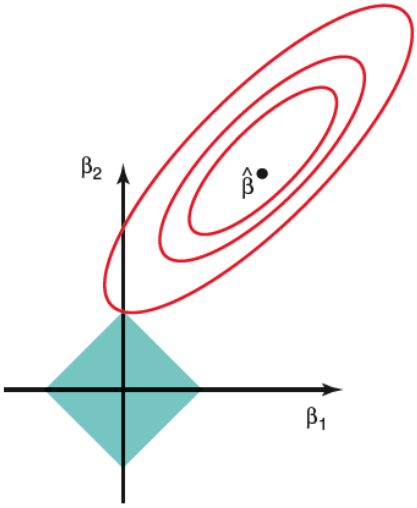
The equivalent constrained minimisation problem applied to our example is:

$$\min_{\beta_1, \beta_2} \quad RSS + \lambda(|\beta_1| + |\beta_2|)$$

$$= \min_{\beta} \quad RSS$$
$$\text{s.t.} \quad |\beta_1| + |\beta_2| \leq s$$

In contrast to Ridge, Lasso's constraint is the shape of a diamond, reflecting a constant marginal benefit of shrinking coefficients, rather than diminishing:

Figure 3: Lasso graphical representation



Due to the straight-line shape of the constraint, the optimal trade-off between minimising RSS and shrinking coefficients can often occur at corner solutions. This illustrates why Lasso will often set some coefficients to zero and is used for model selection.

Overall, deviating from minimising RSS means that Ridge and Lasso are biased. Nonetheless, both often succeed in reducing MSE. By shrinking coefficients, this reduces model flexibility and variance, which can often outweigh the added bias. Ridge shrinks coefficients smoothly, whilst Lasso does so linearly. Therefore, Ridge will not set coefficients to zero, whilst Lasso can.

(b) Building on our previous example, suppose the dependent variable refers to the salary of footballers, whilst the explanatory variables include goals, assists, shots on target, etc. This is purely for illustrative purposes and to support our intuition.

For comparability across methods, the same training and test samples were used. Additionally, we repeat this analysis across five seeds as a robustness check to evaluate the sample-dependency of our results. When observing the data, we noticed variables $X9$-$X11$ took on small, distinct integer values, indicating they represent categorical groupings. These were therefore specified as factors to allow the model to capture differences across categories without imposing an artificial linear structure. $X1$-$X8$ displayed a range of values consistent with continuous variables and therefore remained in numerical form. Next, we prepared our data by forming a matrix of explanatory variables.

To understand the relationship between λ and coefficient estimates, we first generate a wide grid of candidate values for λ, allowing us to compare models ranging from very lenient to very strict. The more lenient ones will be similar to OLS, whilst the stricter ones will severely penalise the square of coefficients, essentially explaining salary with the average value in the training sample (-19.59)[1]. *Figure 4* shows how coefficients vary with the candidate values for λ.

To choose the value of λ, we apply 10-fold cross validation to the training sample. This randomly splits the training sample into ten folds. Assuming a value of λ, a Ridge model is trained on nine folds and MSE is calculated on the remaining one. This is repeated ten times, ensuring each fold is used once as a test fold. This is also repeated for various values of λ. The one with the lowest average MSE on the test folds of the training data is selected.

Table 2 shows the optimal values of λ across the five seeds. The severity of the penalty ranges from very high in seed 3 (1186.23), to very low in seed 5 (13.89). This variation likely reflects sample-specific differences. For example, in seed 3, the players most likely differed more between each fold. To prevent overfitting to specific folds, Ridge chooses a stronger penalty, heavily shrinking the coefficients to perform better on the test folds of the

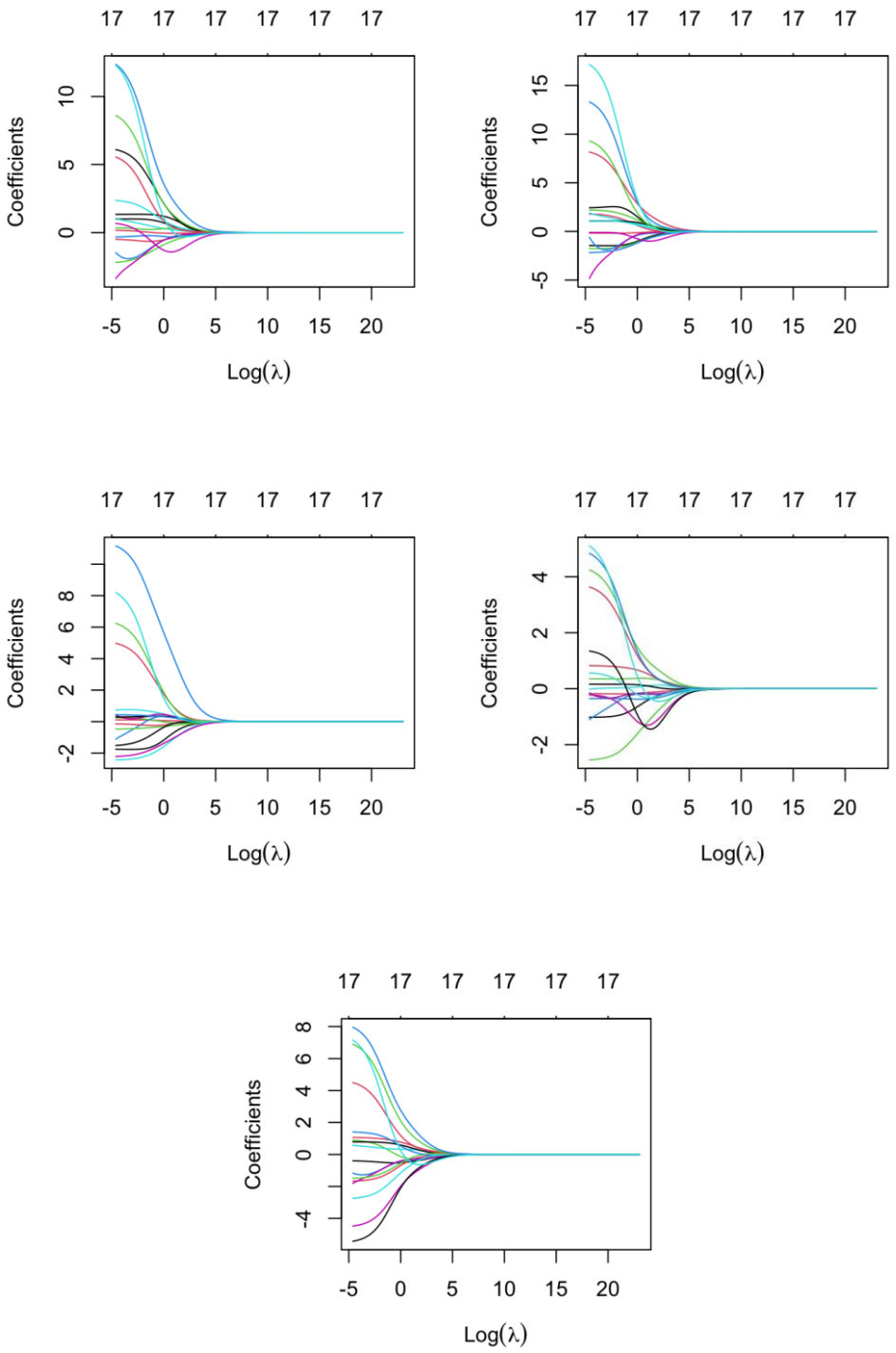
[1] *Note that the predicted value is negative. This is not economically meaningful when interpreted as a salary, but the example is intended for illustrative purposes only.*

training sample. By contrast, for seed 5 the optimal value is much smaller, suggesting the players across each fold had more similar characteristics. *Figure 5* shows the cross-validated error for various $\log(\lambda)$, indicating where the cross-validated error is minimised.

After determining the optimal value of λ in each seed, we use it to estimate a Ridge model on the training data. By plugging each observation in the training and test data into this model, we form predictions and calculate training and test MSE, which are presented in *Table 4*. Ridge consistently has a higher training MSE but a lower test MSE than OLS. For instance, in seed 1, the training MSE for OLS and Ridge are 32.74 and 40.95 respectively. This shows OLS has a lower bias. However, test MSE for OLS and Ridge are 47.63 and 41.31 respectively in seed 1. This suggests Ridge reduced variance sufficiently to offset bias, offering superior performance on the test data.

Finally, we used the optimal value of λ in each seed to estimate a Ridge model on the whole dataset. As seen in *Table 3*, OLS produces larger parameter estimates. In particular $X4$ and $X6$, which have a correlation of 0.98, have large opposing coefficients. Suppose $X4$ and $X6$ represent goals and shots on target respectively. Strong players would produce high numbers of both but perhaps goals are the more important determinant of salary. Hence, OLS may assign a very large, positive coefficient to $X4$ and a very large, negative coefficient to $X6$, to force the regression line through the data points. On the other hand, Ridge shrinks these coefficients. The extent to which Ridge shrinks these coefficients largely depends on differences between training folds when choosing the optimal value of λ. For instance, seed 3 produces extremely small coefficients, whilst they are not as small for seed 5.

Figure 4: Relationship between coefficients and severity of penalty for Ridge[2]



[2] *Logarithms on the x-axes are natural logarithms*

Figure 5: Relationship between average MSE and severity of penalty for Ridge

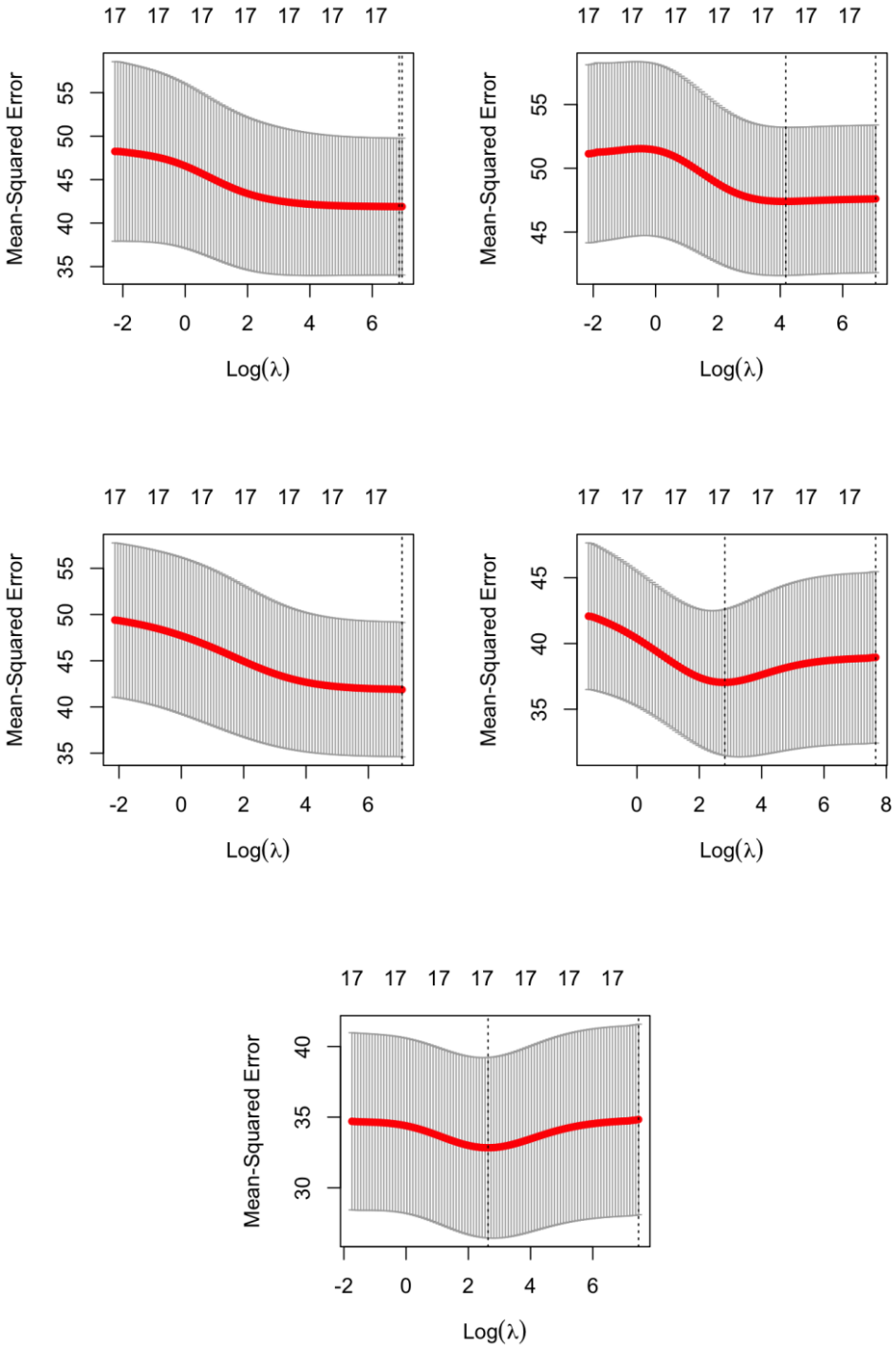


Table 1: Correlation matrix

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	Y
X1	1.00	0.05	0.02	0.84	-0.04	0.83	0.02	-0.05	0.02	0.04	0.77	-0.08
X2	0.05	1.00	0.04	0.02	-0.42	-0.06	0.34	-0.23	0.88	0.06	0.03	0.02
X3	0.02	0.04	1.00	0.08	-0.03	0.07	0.12	0.06	0.03	0.89	0.10	-0.18
X4	0.84	0.02	0.08	1.00	-0.05	0.98	0.03	-0.01	0.01	0.08	0.93	-0.1
X5	-0.04	-0.42	-0.03	-0.05	1.00	0.14	-0.2	0.53	-0.41	-0.08	-0.08	0.07
X6	0.83	-0.06	0.07	0.98	0.14	1.00	-0.01	0.09	-0.07	0.07	0.91	-0.08
X7	0.02	0.34	0.12	0.03	-0.2	-0.01	1.00	-0.1	0.3	0.15	0.03	0.05
X8	-0.05	-0.23	0.06	-0.01	0.53	0.09	-0.1	1.00	-0.26	0.04	-0.07	0.04
X9	0.02	0.88	0.03	0.01	-0.41	-0.07	0.3	-0.26	1.00	0.03	0.03	0.02
X10	0.04	0.06	0.89	0.08	-0.08	0.07	0.15	0.04	0.03	1.00	0.11	-0.17
X11	0.77	0.03	0.10	0.93	-0.08	0.91	0.03	-0.07	0.03	0.11	1.00	-0.05
Y	-0.08	0.02	-0.18	-0.1	0.07	-0.08	0.05	0.04	0.02	-0.17	-0.05	1.00

Table 2: Optimal severity of penalty for Ridge and Lasso

Seed	Ridge	Lasso
1	954.68	1.05
2	65.04	1.16
3	1186.23	1.08
4	16.76	0.58
5	13.89	1.10

Table 3: Parameter estimates for seeds 1-5

Variable	OLS	Ridge (Seed 1)	Ridge (Seed 2)	Ridge (Seed 3)	Ridge (Seed 4)	Ridge (Seed 5)
Intercept	-25.46	-19.15	-19.12	-19.15	-19.12	-19.12
X1	-0.08	0.00	-0.04	0.00	-0.07	-0.07
X2	0.07	0.00	0.01	0.00	0.04	0.05
X3	-1.17	-0.01	-0.10	0.00	-0.28	-0.31
X4	7.14	0.00	-0.04	0.00	-0.08	-0.09
X5	2.52	0.00	0.04	0.00	0.13	0.15
X6	-9.98	0.00	-0.03	0.00	-0.06	-0.06
X7	0.54	0.00	0.03	0.00	0.12	0.14
X8	0.32	0.00	0.02	0.00	0.07	0.08
X9(0)	(Ref)	-0.00	-0.02	0.00	-0.05	-0.06
X9(1)	0.46	0.00	0.02	0.00	0.05	0.06
X10(2)	0.03	-0.00	-0.02	0.00	-0.07	-0.08
X10(3)	-1.11	-0.01	-0.17	0.00	-0.50	-0.56
X10(4)	0.15	-0.02	-0.19	0.00	-0.47	-0.52
X11(2)	4.65	0.01	0.07	0.00	0.20	0.24
X11(3)	6.91	0.01	0.17	0.00	0.51	0.58
Variable	**OLS**	**Lasso (Seed 1)**	**Lasso (Seed 2)**	**Lasso (Seed 3)**	**Lasso (Seed 4)**	**Lasso (Seed 5)**
Intercept	-25.46	-19.15	-19.15	-19.15	-19.09	-19.15
X1	-0.08	0.00	0.00	0.00	0.00	0.00
X2	0.07	0.00	0.00	0.00	0.00	0.00
X3	-1.17	-0.11	0.00	-0.08	-0.57	-0.06
X4	7.14	0.00	0.00	0.00	0.00	0.00
X5	2.52	0.00	0.00	0.00	0.00	0.00
X6	-9.98	0.00	0.00	0.00	0.00	0.00
X7	0.54	0.00	0.00	0.00	0.00	0.00
X8	0.32	0.00	0.00	0.00	0.00	0.00
X9(0)	(Ref)	0.00	0.00	0.00	0.00	0.00
X9(1)	0.46	0.00	0.00	0.00	0.00	0.00
X10(2)	0.03	0.00	0.00	0.00	0.00	0.00
X10(3)	-1.11	0.00	0.00	0.00	0.00	0.00
X10(4)	0.15	0.00	0.00	0.00	0.00	0.00
X11(2)	4.65	0.00	0.00	0.00	0.00	0.00
X11(3)	6.91	0.00	0.00	0.00	0.26	0.00

Table 4: MSE for OLS, Ridge and Lasso for seeds 1-5

Seed 1			
	Train	**Test**	**Delta**
OLS	32.74	47.63	14.89
Ridge	40.95	41.31	0.36
Lasso	41.03	41.34	0.31

Seed 2			
	Train	**Test**	**Delta**
OLS	35.85	50.05	14.19
Ridge	45.53	36.04	-9.49
Lasso	46.72	36.00	-10.71

Seed 3			
	Train	**Test**	**Delta**
OLS	35.18	46.28	11.10
Ridge	40.76	41.25	0.49
Lasso	40.55	41.29	0.75

Seed 4			
	Train	**Test**	**Delta**
OLS	30.12	50.43	20.31
Ridge	33.68	45.05	11.38
Lasso	32.44	47.10	14.67

Seed 5			
	Train	**Test**	**Delta**
OLS	25.87	53.96	28.08
Ridge	29.81	49.14	19.34
Lasso	32.22	48.62	16.40

(c) To understand how Lasso behaves, we create a grid of candidate values for λ. We estimate Lasso models and plot how the coefficients evolve with these candidate values (*Figure 6*). For very small λ, we get a spread of coefficients, which, based on the loss function, we know will approximate OLS. As λ grows, coefficients are shrunk, until all coefficients are 0. As such, very small λ would not improve upon OLS, and very large λ results in predicting the mean of Y in the training sample.

Once again, we applied 10-fold cross validation to the training data to determine the optimal λ in each seed, following the same process outlined in (b). *Figure 7* shows the cross-validated error for various $\log(\lambda)$, indicating the one which minimises average MSE on the test folds of the training data. Similarly, each seed chooses a different value, based on the noise between folds. These are shown in *Table 2*.

Despite the optimal λ being found using different training samples across seeds, there is a clear pattern. Values consistently landed just above one, except for seed 4, which selected 0.58. Given the consistency of the other four seeds, we argue this difference is primarily due to specific features of the training sample for seed 4. For example, in randomly splitting this training sample, we may have ended up with folds that shared a great deal of characteristics (lacked significant noise). As a result, a large λ may not have been necessary in minimising cross-validated error. This suggests our dataset may be following strong general patterns with a moderate amount of variation in player characteristics.

Next, we use the optimal value of λ for each seed to estimate a Lasso model using the training data. From this, we generate predictions for all observations in the training and test sample, allowing us to calculate their respective MSE, as shown in *Table 4*. We can see that Ridge and Lasso consistently have a higher training MSE than OLS, as expected. OLS is unbiased and designed to be the linear estimator that most accurately fits the training data. Lasso, however, is also linear, but deliberately biased. Its power comes from reducing model complexity (see *Figure 1: The Bias-Variance Trade-Off*). Across the seeds, this works, with Lasso outperforming OLS on test MSE, implying that penalising coefficients has produced a model that more accurately predicted unseen data. In other words, OLS is

modelling features of the training sample players so well that is fails to generalise. Lasso partially restrains its estimates resulting in more accurate out-of-sample predictions.

Comparing the predictive performance of Lasso and Ridge is more nuanced. Both significantly improve test MSE relative to OLS, but relative to each other, the difference is unclear. Ridge performs extremely similarly to Lasso, only marginally outperforming it on test MSE in three out of five seeds. One possible explanation for Ridge's slight advantage is the presence of strong multicollinearity between several explanatory variables, notably $X4$, $X6$, and $X11$ ($X4$ has correlations 0.98 and 0.93 with $X6$ and $X11$, respectively). When predictors are highly correlated, if each contains some relevant information, Ridge can perform better on unseen data because it shrinks coefficients smoothly, retaining their combined contributions. Lasso, on the other hand, may select more dominant signals effectively removing the others from the model rather than sharing their effects. If multiple correlated variables are jointly informative, Lasso can lose some of this predictive ability, which may explain why Ridge slightly outperformed on this dataset. However, with our current data, the difference is small and this cannot be argued with certainty. Overall, it is very clear that both shrinkage methods have improved upon OLS in predicting out-of-sample data.

Finally, using the optimal λ, we estimated a Lasso model on the whole dataset. The coefficients listed in *Table 3* show that OLS produces the more flexible model. Each variable is assigned a non-zero coefficient, as expected. Lasso, across all seeds, sets many of these to 0, removing them from the model. $X3$, however, rarely has its coefficient set to 0. Essentially, Lasso is mostly predicting with only an intercept and $X3$. This highlights the contrast between Ridge and Lasso. Ridge distributes effects, by shrinking smoothly. Lasso shrinks linearly, selecting specific variables. Note that $X4$ and $X6$, with the largest OLS coefficients, have been removed by Lasso in all seeds. Choosing X3 is interesting. It has very weak correlation with all variables, except 0.89 with X10. X3 and X10 have incredibly similar correlations with Y. Lasso may be identifying a unique signal from X3/X10, but then selects just one, depending on slight sample differences. These results may indicate that the predictive structure of the data is driven by a small number of dominant signals, and other correlated variables are not adding enough explanatory power to justify their inclusion.

Figure 6: Relationship between coefficients and severity of penalty for Lasso

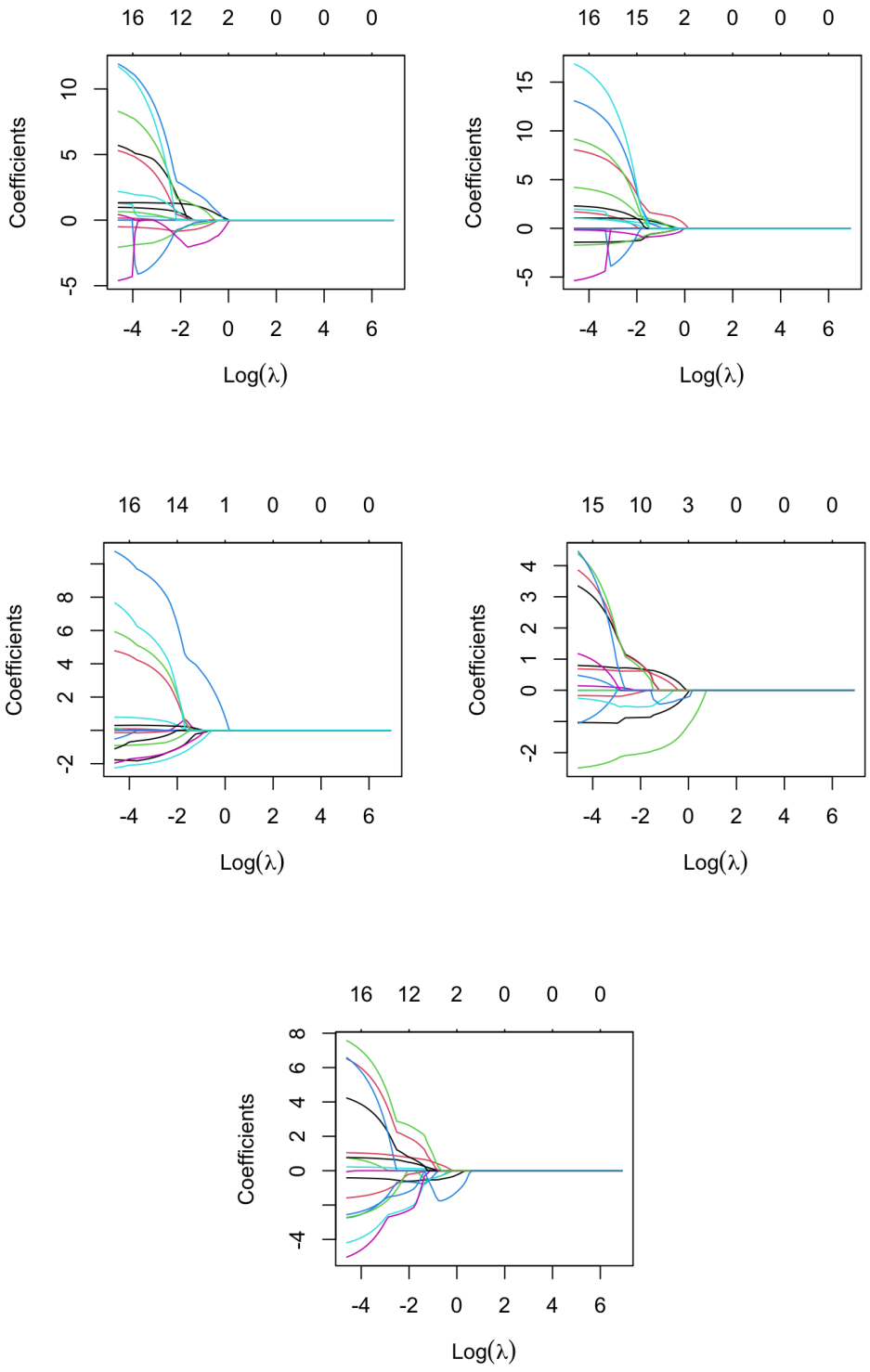
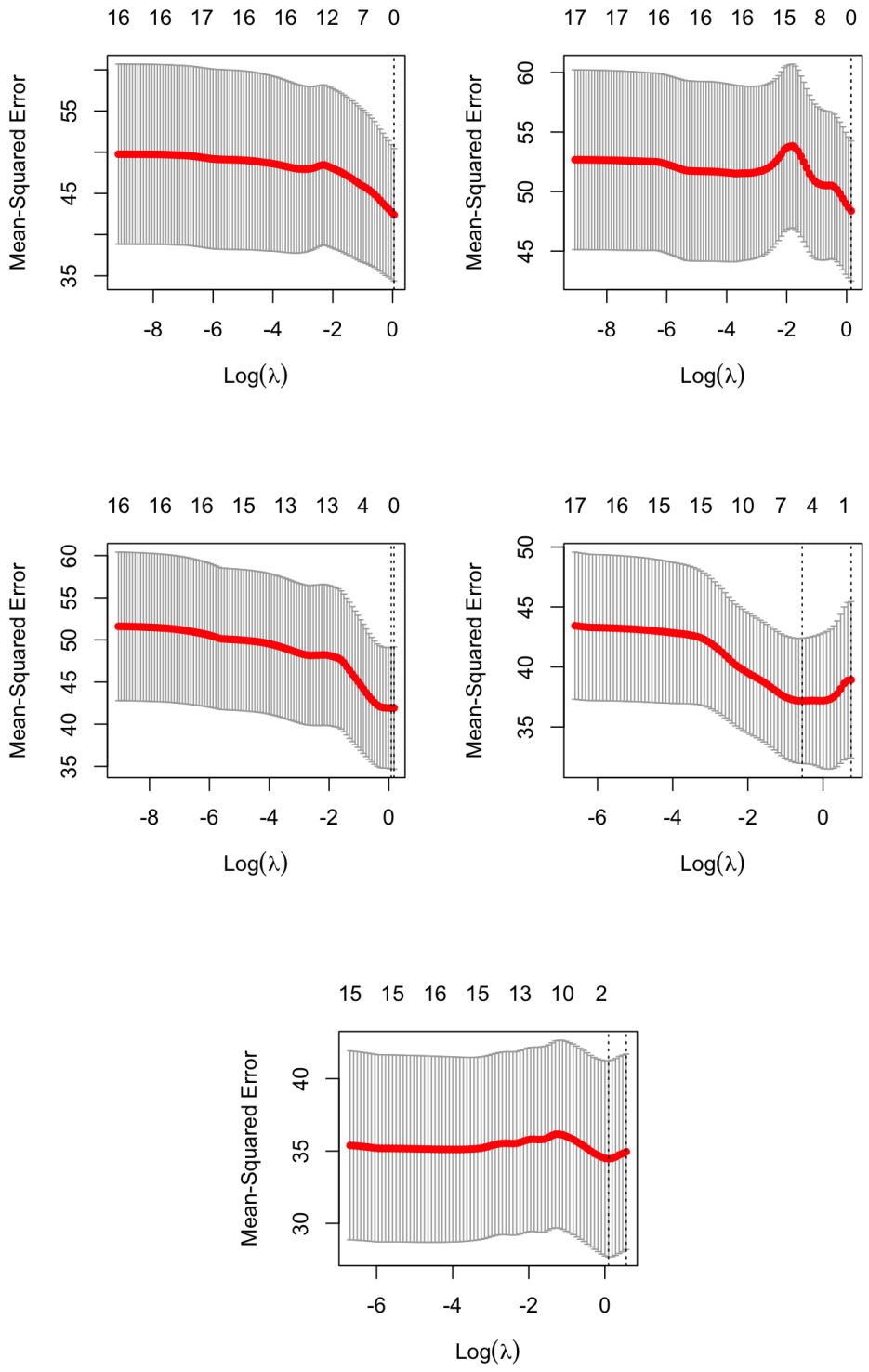


Figure 7: Relationship between average MSE and severity of penalty for Lasso



2. Tree and ensemble methods

(a) Regressions trees are an alternative to OLS and are most suitable for data with nonlinear logic. Using the same example as before, we consider building a tree to predict the salary of footballers. The tree partitions the sample space into J regions, where the prediction in each region is the average salary. Ideally, this would be done to minimise the MSE on the training sample. However, there are so many ways to split these 11 variables into J regions. Searching for the smallest MSE across all possible splits for all values of J would take an extremely long time, making it computationally infeasible.

Instead, we use the 'greedy top-down approach'. This starts by calculating MSE_0, which is the MSE from predicting all players have the average salary. To determine the first split and form the 'root node', the algorithm searches across all 11 explanatory variables for the split which yields the largest reduction in MSE relative to MSE_0. After this initial split, the algorithm is repeated in each region. Further splits are implemented until no further reductions in MSE are possible, meaning we have reached a 'leaf'.

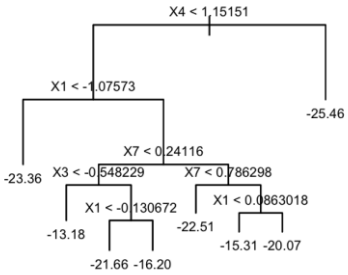
To begin, we used the training data to estimate deep trees (trees using all explanatory variables) using the greedy top-down approach. We required a minimum of 10 observations in each region to prevent overfitting. Whilst the training data features scaled explanatory variables, this is not strictly necessary for growing trees. For instance, if scoring 10 goals last season was used for the initial split, all players would stay in the same group after scaling.

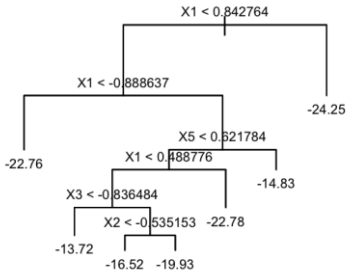
To illustrate how the sample space is partitioned, *Figure 8* shows trees relating Y to X1-X11. For seed 1, the initial split uses X4 and a value of 1.15. Suppose X4 is years of experience. Splitting players into one group with less than 1.15 years of experience and another with 1.15 or more and predicting the average salary for each group, results in the largest reduction in MSE. For more experienced players, no further splits are made, meaning it was not possible to further reduce the MSE on that branch. However, the MSE could be improved by further splitting the less experienced players based on X1, with a

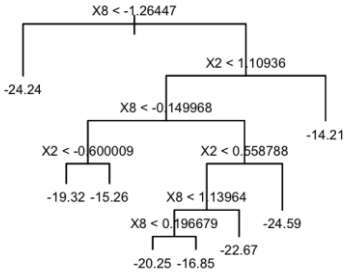
value of -1.08, and many further splits occurred for the less experienced players for whom $X1 \geq -1.08$, suggesting these players are more heterogenous.

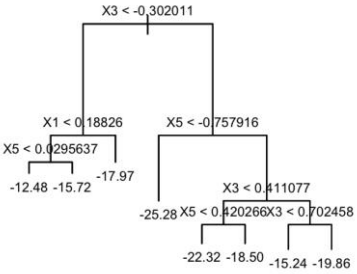
Whilst the greedy top-down approach is useful, it is prone to overfitting. This is because reducing MSE with further splits can lead to the tree copying specific characteristics of the training data. *Figure 8* indicates this may be happening in our case. For instance, almost all trees use a different variable for the initial split, and many further splits are also carried out using a range of variables. It appears they are struggling to notice consistent patterns and are instead mimicking patterns from the training sample.

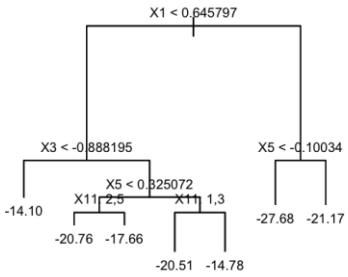
Figure 8: Deep trees for seeds 1-5











To counteract overfitting, we use complexity pruning, which cuts leaves off the deep trees. This can reduce the MSE on the test sample by reducing variance, despite introducing more bias. The pruned trees aim to minimise an alternative loss function: MSE and a penalty proportional to the number of leaves (MSE + αL). This determines the pruned trees which best succeed in balancing the incentives to predict well and remain simple. The algorithm assumes an initial severity of the penalty, α, then calculates the cross-validated loss function for all sub-trees formed by cutting off one leaf. This is repeated for different values of α, with the tree and α which minimise the cross-validated loss function on the test folds of the training data being selected. This algorithm is repeated for the sub-trees which can be formed by dropping more leaves. *Figure 9* plots the outcome of this process, showing the trade-off between predictive accuracy and number of leaves. For each seed, we estimated a pruned tree using the number of leaves minimising this trade-off (see *Figure 10*). The seeds missing from this figure produced stubs, where predictions are the average of Y in the training sample.

Table 5: Comparison of train and test MSE for deep trees

Metric	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**	**Seed 5**
Train MSE	23.92	32.62	26.77	23.79	17.93
Test MSE	49.47	39.58	55.48	57.38	55.80
Delta	25.55	6.97	28.71	33.59	37.87

Table 5 shows the deep trees did appear to be overfitting, with the MSE on the test data fluctuating and consistently higher.

Table 6: MSE for OLS vs. pruned tree

	Seed 1		Seed 2	
	OLS	**Tree**	**OLS**	**Tree**
Training	32.74	41.03	35.85	42.30
Test	47.63	41.34	50.05	38.93
Delta	14.89	0.31	14.19	-3.37

Seed 3		Seed 4		Seed 5	
OLS	**Tree**	**OLS**	**Tree**	**OLS**	**Tree**
35.18	40.79	30.12	38.31	25.87	23.90
46.28	41.27	50.43	44.07	53.96	52.30
11.10	0.48	20.31	5.76	28.08	28.40

Table 6 shows pruned trees have a lower test MSE than deep trees and exhibit limited overfitting. Whilst OLS has a consistently lower training MSE, the pruned trees outperform OLS on the test data. OLS is more flexible, allowing it to model characteristics of the training data. Whilst this succeeds in minimising bias, it can produce volatile predictions. On the other hand, the pruned trees are highly inflexible, using either the average of Y in the training sample or 1-2 explanatory variables. This adds bias but reduces variance, improving predictive accuracy in our case.

Figure 9: Relationship between predictive accuracy and complexity

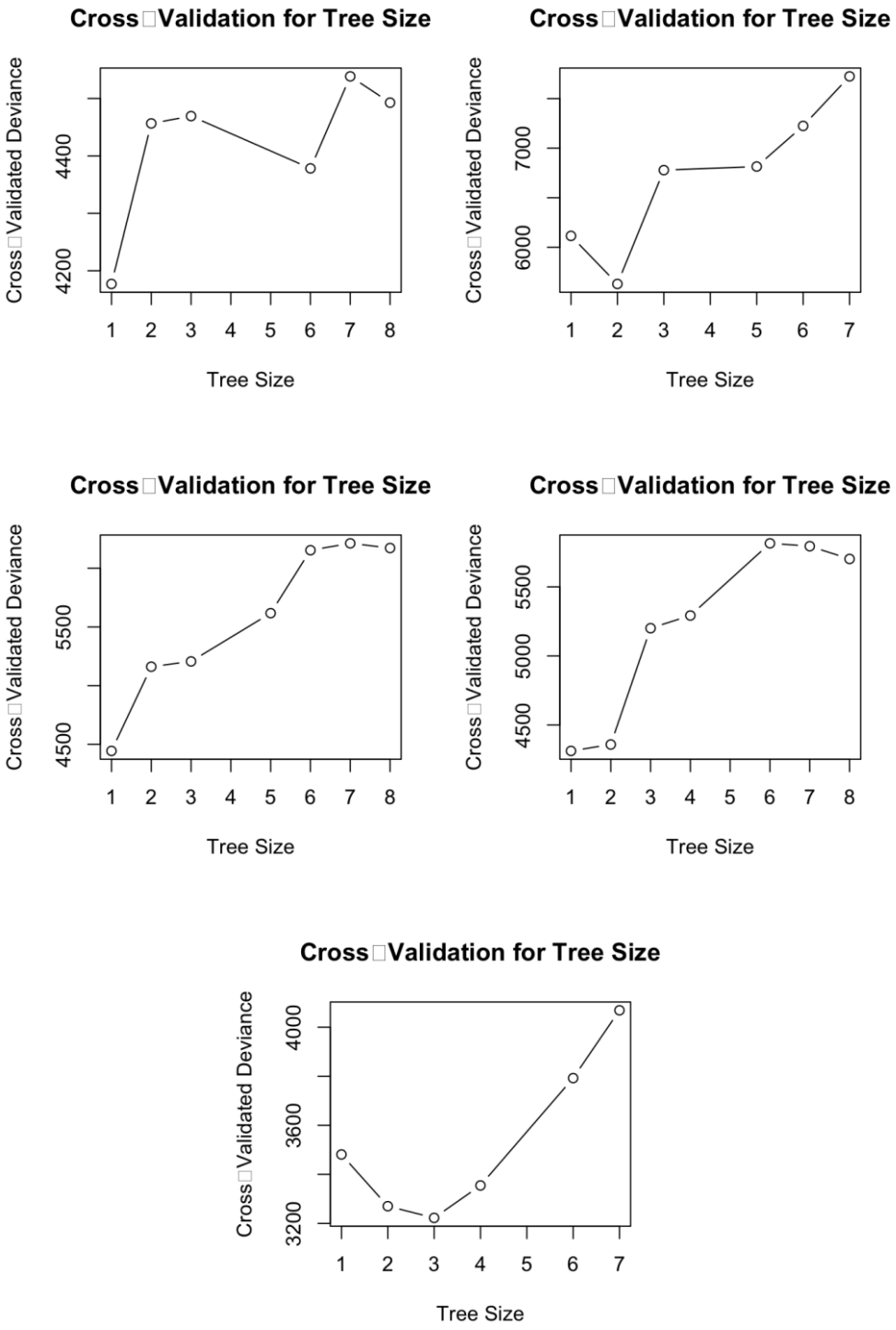
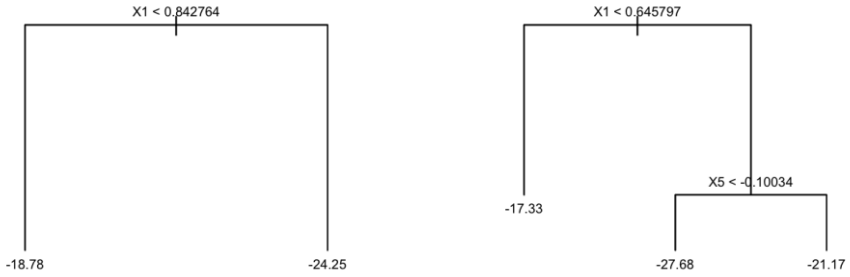


Figure 10: Pruned trees for seeds 2 and 5



(b) Different training samples can produce very different trees because the greedy top-down approach often overfits. Ensemble methods, such as bagging, random forest (RF) and gradient boosting (GB), are designed to outperform trees by lowering variance. This is done using many trees to form predictions, much like forming a consensus. Suppose we grow m trees to predict the salaries of footballers and each tree has variance σ^2. If we could grow independent trees, our ensemble would have a variance of $\frac{\sigma^2}{m}$, improving predictive accuracy.

Bagging aims to reduce variance by using many de-correlated deep trees to form predictions. In our case, we estimate 500 trees using the training data. However, we do not use all the observations from the training data to estimate each tree. Instead, they are estimated using bootstrap samples. These are formed by randomly selecting observations with replacement, meaning the same observations can appear multiple times. This aims to make the samples less similar and less correlated. Once the required number of observations is met, this is repeated until there are 500 bootstrap samples, with each one being used to train a deep tree. Once our model has been estimated, we form predictions by passing observations through the ensemble, generating 500 predictions per observation. The average prediction is assigned to each observation. After doing this to the training and test samples, we calculate their respective MSE.

The deep trees used in bagging have very low bias. *Table 7* shows a very low training error for bagging, reflecting potential overfitting. Despite this, bagging outperforms OLS and the pruned tree on the test data. This is likely because OLS has such high variance, whilst the pruned tree has high bias. Bagging appears to be striking a favourable balance between bias and variance for predicting on this dataset.

As bagging can be confusing, we form variable importance plots (*Figure 11*) to illustrate overfitting. These are generated with the unused observations from each bootstrap sample, known as 'out of bag' (OOB) observations. For instance, suppose you have a training sample of 100 footballers. If you used 63 players in a bootstrap sample, 37 were not used to estimate the corresponding tree model. We take these 37 players and pass them through the tree corresponding to the bootstrap sample they were left out of. This will form

37 predictions of salary, from which we calculate OOB MSE. We repeat this for all trees, taking the average OOB MSE. The '%IncMSE' plots tell us how much this increases when certain variables are no longer used, showing us how useful each variable is. The composition of variable importance is seed-dependent and relatively top-skewed, meaning each ensemble depends on the features of the training sample. Therefore, it could be possible to improve upon the bagging model by counteracting overfitting.

This is what RFs aim to do. These are very similar to bagging but with one crucial difference designed to further reduce variance. Rather than freely estimating deep trees, RFs limit each tree to a random subset of explanatory variables. This is because certain features might be related to specific variables, resulting in them dominating the deep trees and producing high correlations between each tree. For instance, the most important determinant of football player salary might be goals. Whilst this variable could be useful for building an ensemble to predict salary, predictive accuracy could vary massively depending on the composition of strikers in the test sample.

For our RF model, we limit our random subset to four because p/3 is typically low enough to ensure certain variables are not constantly dominating the trees (Hastie, Tibshirani and Friedman, 2009). For instance, if X5 is dominant but a tree can only form splits using X1-X4, this allows it to pick up on other details. Once again, bootstrapping is used and 500 trees are estimated. Similarly, we pass the observations from both samples through the ensemble to estimate their respective MSE.

Table 7 shows RF has a slight edge over bagging in terms of test MSE. The variable importance plots in *Figure 12*, estimated the same way as before, help us understand why. Much like bagging, variable importance is seed dependent. The distribution of importance is still top-skewed but now much smoother. This means RF has reduced overfitting, as reflected by the slightly higher training MSE. As such, RF has a lower variance than bagging, often allowing it to perform better on the test data. Therefore, RF also succeeds in outperforming OLS and the pruned tree.

GB is very different to the other methods. Rather than growing many trees and taking the average prediction, this method grows trees sequentially to improve upon the last. We start with a simple prediction, F_0. In our case, this is the average of Y in the training sample. This associated residual is $res_1 = Y - F_0$. Next, we grow a tree using the training sample with this residual as the dependent variable, limiting the tree to four leaves to prevent overfitting. For each observation in the training sample, this tree predicts the residual, generating h_1. Low values of $h_1$are associated with observations which look like they should be close to the sample average based on the explanatory variables. These predicted residuals are used to form a new prediction of Y, $F_1 = F_0 + \nu h_1$, where ν is the learning rate. In other words, we are predicting all footballers in the training sample have the same salary, then partially accounting for how wrong we expect this to be. Typically, the learning rate is low because we want to follow underlying patterns, not overcorrect to outliers, so we use 0.1. Using our new prediction, F_1, we produce the residual $res_2 = Y - F_1$. We estimate another tree in the same way but using res_2 as the dependent variable. This produces the expected error from our second attempt at predicting Y, h_2. We use this to form a new prediction of Y, $F_2 = F_1 + \nu h_2$. This process is repeated until we have estimated B trees, producing a prediction of Y using the model $F_B = F_{B-1} + \nu h_b$. Since $F_{B-1} = F_{B-2} + \nu h_{B-1}$, we get:

$$F_B = F_0 + \nu h_b + \nu h_{b-1} + \nu h_{b-2} + \ldots + \nu h_1$$

Therefore, our final model is claiming: "we made an initial prediction and adjusted it many times based on how wrong we expected the preceding prediction to be".

To determine the number of trees which minimises MSE, we apply 5-fold cross validation across 200 trees to the training sample, producing *Figure 13*. The blue dashed line indicates where the cross-validated MSE is minimised. Whilst this is sample-dependent, it is usually less than 50 trees, with more resulting in overfitting. Our final model is estimated using the optimal number of trees. Observations in both samples are passed through each tree, sequentially improving predictions until we get our final predictions. From this, we calculate train and test MSE.

Overall, GB models are very different to bagging and RF but they all aim to form high quality predictions using many uncorrelated trees to reduce variance. *Table 7* shows GB has a higher training error than bagging and RF, indicating less overfitting, which will further increase bias but lower variance. GB sometimes outperforms bagging and RF on test MSE, indicating the lower variance is often beneficial but not always. Similarly, GB also outperforms OLS and the pruned tee with a more suitable mix of bias and variance.

Table 7: Comparison of train and test MSE for seeds 1-5

Seed 1				
Model	**Train**	**Test**	**Delta**	**Rank**
OLS	32.74	47.63	14.89	5
Tree	41.03	41.34	0.31	3
Bagging	5.84	40.72	34.87	4
Random Forest	6.50	38.59	32.09	2
Gradient Boosting	16.86	37.90	21.04	1
Seed 2				
Model	**Train**	**Test**	**Delta**	**Rank**
OLS	35.85	50.05	14.19	5
Tree	42.30	38.93	-3.37	4
Bagging	7.16	37.18	30.02	3
Random Forest	7.71	34.91	27.20	1
Gradient Boosting	22.65	36.33	13.68	2
Seed 3				
Model	**Train**	**Test**	**Delta**	**Rank**
OLS	35.18	46.28	11.10	5
Tree	40.79	41.27	0.48	4
Bagging	7.56	33.14	25.58	1
Random Forest	8.08	33.35	25.27	2
Gradient Boosting	36.15	38.24	2.08	3
Seed 4				
Model	**Train**	**Test**	**Delta**	**Rank**
OLS	30.12	50.43	20.31	5
Tree	38.31	44.07	5.76	2
Bagging	6.74	46.56	39.82	3
Random Forest	7.18	42.85	35.67	1
Gradient Boosting	18.70	46.57	27.87	4
Seed 5				
Model	**Train**	**Test**	**Delta**	**Rank**
OLS	25.87	53.96	28.08	5
Tree	23.90	52.30	28.40	4
Bagging	4.32	46.64	42.32	1
Random Forest	4.37	46.98	42.61	2
Gradient Boosting	9.10	49.02	39.92	3

Figure 11: Variable importance for bagging models

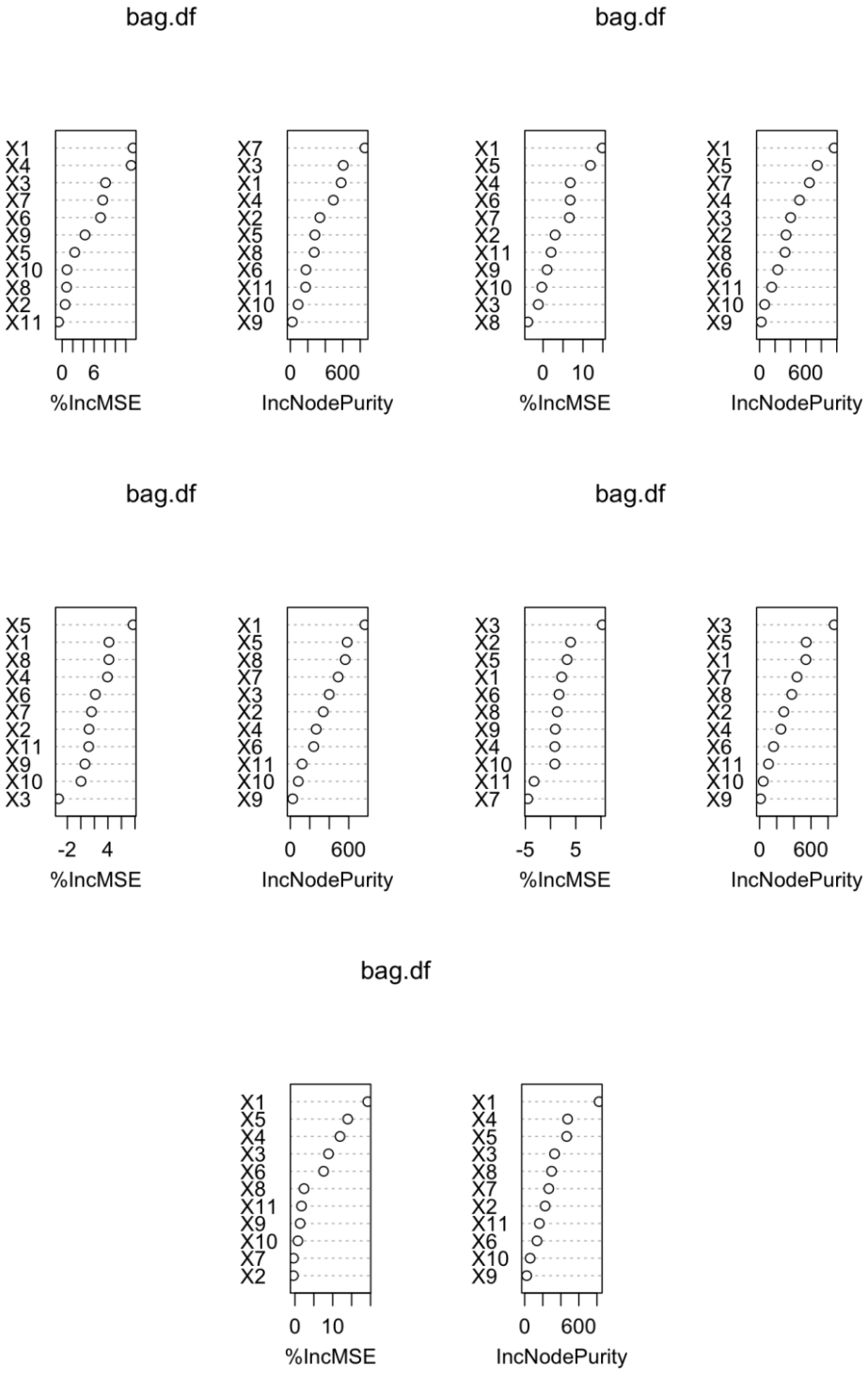


Figure 12: Variable importance for random forest models

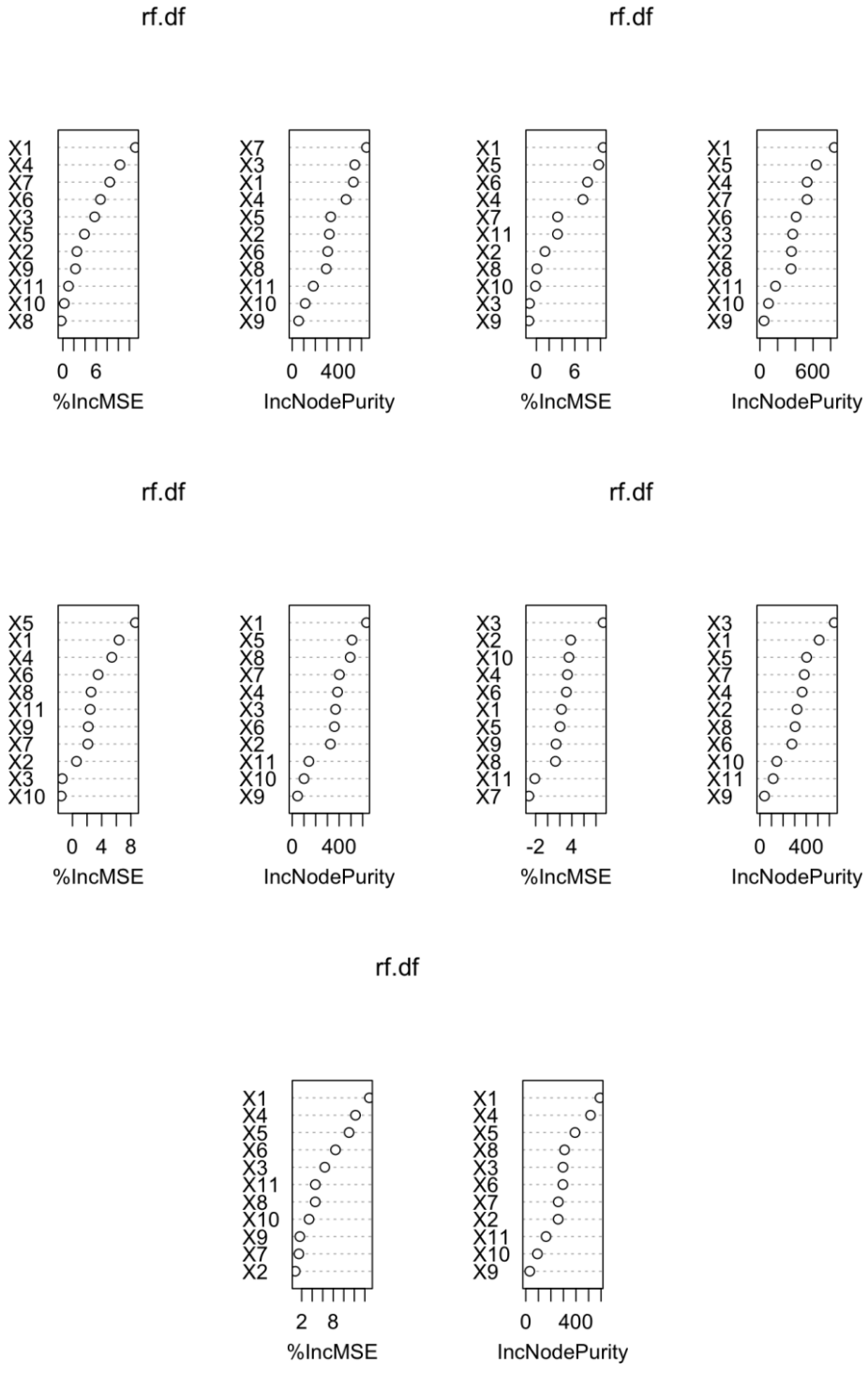
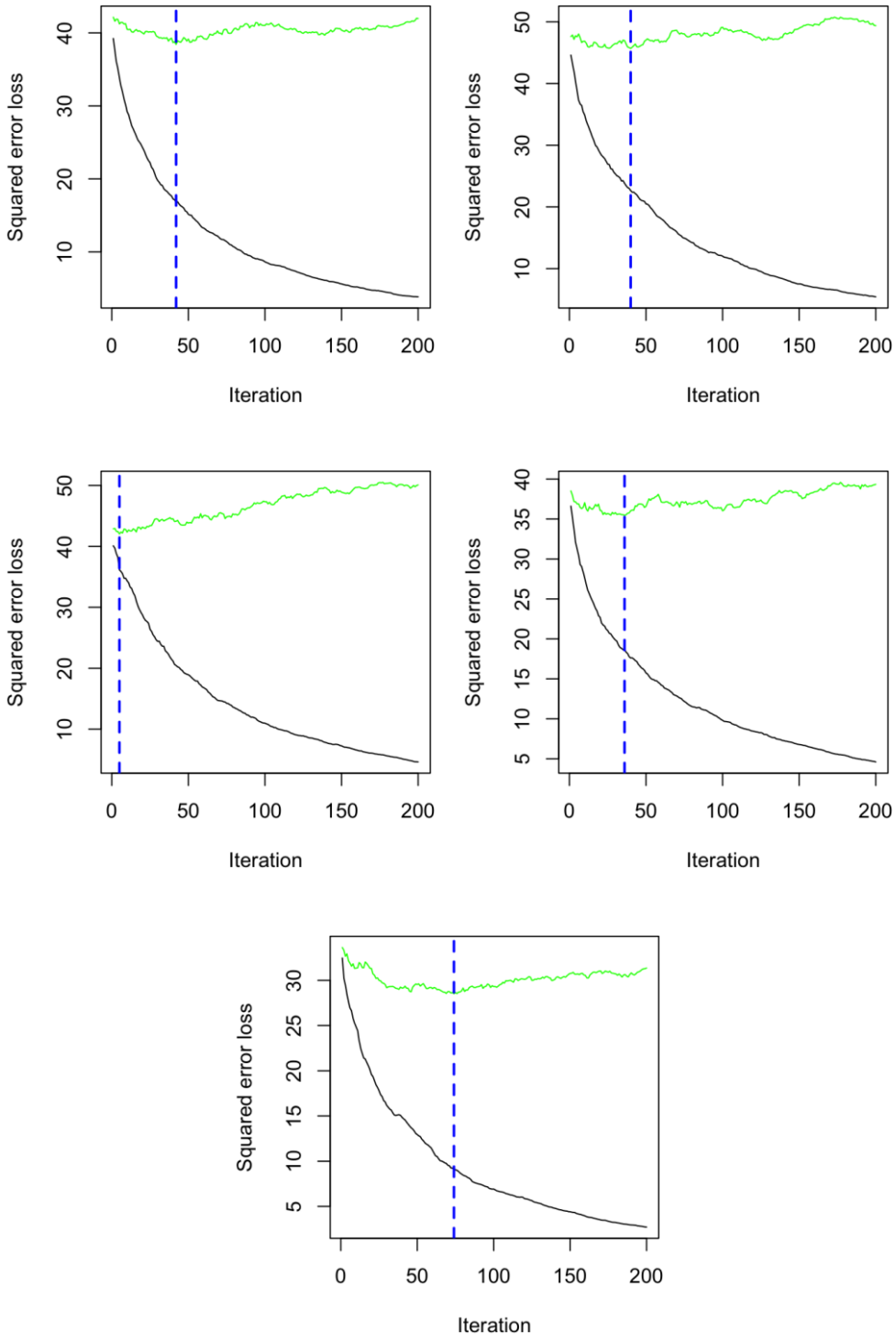


Figure 13: Relationship between cross-validated MSE and number of trees



ES32039 coursework – part 2

1. Introduction

In Part 2, we aim to predict whether someone has high income using several characteristics. We do this using a range of classification techniques. There are two main goals of this exercise. Firstly, we aim to form an understanding of the nature of the decision boundary (DB) for income level. Secondly, we aim to determine which are the most important predictors of high income.

To ensure our results and conclusions are not sample-dependent, we conduct our analysis across five arbitrary seeds. The robustness of our results are then further evaluated across an alternative validation set split. Finally, the robustness of our results are also evaluated using gradient boosting (GB) and shallow neural networks (SNN). This is to ensure we do not overlook hidden complexities of the DB by applying simpler, more interpretable methods in the main methodology.

2. Data preparation

First, we searched for missing observations, finding there were none. Therefore, we proceeded with recoding education, marital status, occupation, race, age and hours worked into broader categories. Additionally, we searched for observations with unusual values and dropped them from the dataset. For instance, we identified 2 observations with 'Married AF-Spouse', which were dropped because this did not fall into any of the categories outlined. We then converted these transformed variables into unordered factors. This ensures each method interprets these variables as qualitative differences, rather than applying a linear interpretation. For example, considering 'was married' a different relationship status to 'never married' makes far more sense than assuming 'was married' is three times higher than 'never married'. Moreover, we felt this qualitative interpretation was more suitable for income and sex, so they were both recoded into factors with two levels. The transformed variables are summarised in the table below.

Raw variable	Transformed variable	Description	Economic interpretation
income	income_level	A factor with 2 levels, indicating whether someone earns more than $50,000 per year.	We want to predict whether someone has high earnings.
education_num	ed_level	A factor with 3 levels relating to the number of years of education.	Can education level help us predict earnings?
marital_status	mar_stat	A factor with 3 levels for never, currently and previously married.	Is relationship status a good predictor of earnings?
occupation	skill_level	A factor with 3 levels, where certain jobs are grouped into different skill levels.	Does the difficulty of a job help us predict earnings?
race	race_transform	A factor with 3 levels for 'white', 'black' and 'other'.	Is race a useful predictor of earnings?
sex	sex_num	A factor with 2 levels for male and female.	Is sex a useful predictor of earnings?
age	age_group	A factor with 5 levels, ranging from young adults to retirement age.	Does age group help us predict earnings?
hours_per_week	hours	A factor with 3 level for part-time, full-time and over-time.	Are hours worked a useful predictor of earnings?

Finally, the dataset was randomly divided into training and test samples using a 70:30 split. We chose this validation set split because 4,000 observations is not a large dataset. We felt this would provide sufficient information for methods which thrive on larger amounts of data – such as random forest (RF), GB and SNN – whilst ensuring the test sample is still substantial enough to evaluate predictive accuracy. However, we also evaluate the robustness of our results using a 50:50 validation set split to determine the stability of the models and ensure our initial results are not a symptom of overfitting.

It is also important to note that we did not apply scaling to our dataset because all explanatory variables are unordered factors. Subtracting the mean and dividing by the standard deviation would strip the useful qualitative interpretation outlined above, instead applying a meaningless numeric scale to each variable.

3. Methodology and results

This section outlines each step we carried out. Our approach involved the application of standard classification techniques, classification trees, bagging and RF. This is because standard classification techniques, such as logistic, are straightforward to interpret, while on the other hand, a nonlinear DB might be more suitable for this data. By comparing how these methods perform, we are hoping to gain insights into the nature of the DB. Additionally, bagging and RF are useful for highlighting which variables have the highest level of predictive power. To ensure our results are not sample dependent, we evaluated them across 5 seeds.

3.1. Standard classification techniques

We first applied standard classification techniques, namely logistic regression, linear discriminant analysis (LDA), quadratic discriminant analysis (QDA), and Naïve Bayes. These are the simplest models which can be used to analyse this question, offering straightforward interpretation. Note that we specifically decided against the use of k-nearest neighbours because it relies on Euclidian distance to determine the similarity between observations. This method could lead to misleading results because the numbers only indicate qualitative differences.

Logistic regression assumes the log odds of having high income are linear in the explanatory variables. This will produce a linear DB. However, we decided against using all the explanatory variables because more flexible models often have a higher variance and worse predictive accuracy. Instead, we estimated a model which assumes the log odds of having high income are linear in age group, education level, marital status, race and skill level. This is because there is robust evidence to suggest these are the most important predictors of income level (outlined in greater detail in *3.3*). We then plugged each observation in the test data into this model, producing predicted probabilities for high income. If the predicted probability exceeded 0.5, we predicted high income, and low income otherwise. This allowed us to calculate the prediction failure rate (PFR) for logistic regression, presented in *Table 8*.

We followed a similar approach when applying the Bayesian models – LDA, QDA and Naïve Bayes. Bayesian models predict high income when the posterior probability of high income is at least greater than that of low income. In plain English, they predict high income when its likelihood, weighted by how common it is, exceeds the likelihood of low income, also

weighted by how common it is. Although we observe how common high and low incomes are, we do not observe the distribution which explains their likelihoods in terms of the explanatory variables. Therefore, each method makes slightly different assumptions about these distributions to form predictions. This results in various DB shapes. LDA assumes the probability density functions for high and low income are normally distributed in terms of the explanatory variables, and their variances are the same. On the other hand, QDA assumes their variances are different. However, Naïve Bayes only assumes the explanatory variables are independent. Following each of these assumptions in turn, we estimated models to predict whether someone has a high income using the same explanatory variables as before. We then plugged the observations from the test sample into each of these models, producing predictions for income level and calculating the PFR. These results are presented in *Table 8*.

Table 8: PFR on test data for standard classification techniques

Model	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**	**Seed 5**
Logistic	0.173	0.180	0.175	0.166	0.193
LDA	0.172	0.191	0.175	0.168	0.208
QDA	0.203	0.209	0.224	0.225	0.213
NB	0.174	0.181	0.175	0.166	0.193

Overall, the best performing technique when using the strongest predictors for income level appears to be logistic regression. Although Naïve Bayes performs similarly, logistic regression is preferred because its interpretation is more straightforward. Therefore, we initially assume a linear DB in terms of the most important explanatory variables is the most suitable fit for the data. This suggests the probability of high income is changing incrementally in variables like marital status, skill level, age, etc.

3.2. Tree methods

Next, we applied tree methods, which are more useful for nonlinear data. If trees outperform logistic, this suggests our initial interpretation of the DB is less suitable and a nonlinear one might be a better fit. For instance, the number of predictions for high income could substantially increase when certain characteristics are met, such as being married or having a high skilled job.

We first estimated a deep tree which related income level to the explanatory variables. This involved running an algorithm which aims to choose splits to maximise ‘node purity’. In

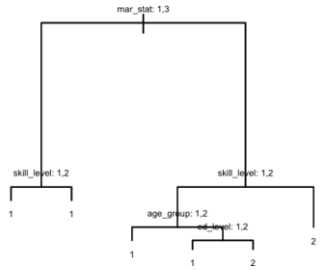
plain English, this means the algorithm searches for ways to separate people with different characteristics, forming two groups where the people in each group are as similar as possible in terms of income level. The algorithm repeats this process until it is not possible to choose any further splits which increase the similarity of people in each group, at which point we have reached a 'leaf'. We predict the most common observation in the training data for each leaf. For instance, *Figure 14* shows '`mar_stat: 1, 3`' was at the root node of all deep trees. This suggests the most effective initial split was to separate married and unmarried people. Thereafter, high-skilled workers were separated from everyone else within these subgroups. However, for unmarried individuals, this created two groups dominated by low earners, which indicates potential overfitting. This is because trees often form redundant splits when they are adopting specific characteristics of the training data, rather than generalising.

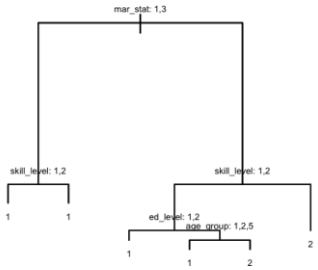
Given signs of overfitting, we applied 'complexity pruning'. This process cuts leaves off to form sub-trees, with the aim of reducing overfitting. Each sub-tree was chosen using a loss function which measures its ability to balance predictive accuracy with a penalty proportional to the number of leaves to discourage overfitting. To do this, we ran an algorithm which assumes an initial severity of this penalty. Cross validation is then used to determine which tree with one less leaf is most effective at balancing this trade-off. After repeating this for various severities of the penalty, the sub-tree and severity of the penalty which best balance this trade-off are selected. This process is repeated for the sub-trees with even fewer leaves. *Figure 15* plots the outcome of this exercise. We can see that the trade-off between predictive accuracy and the number of leaves is minimal at either 3 or 5 leaves depending on the seed. We decided to proceed with the best 3-leaf tree because this is more likely to prevent the tree model from making additional splits which only serve to copy patterns in the training data. However, as seen in *Table 9*, logistic regression outperforms the pruned tree.

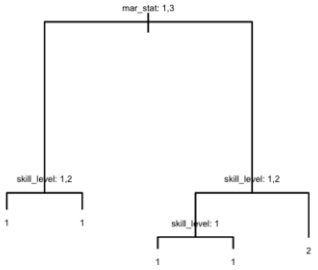
Table 9: PFR on test data for logistic vs. pruned tree

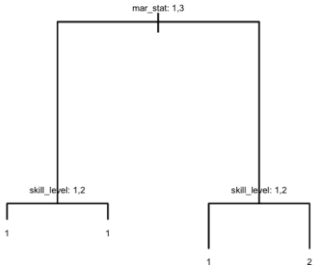
Model	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**	**Seed 5**
Logistic	0.173	0.180	0.175	0.166	0.193
Pruned tree	0.184	0.195	0.192	0.178	0.215

Figure 14: Deep trees









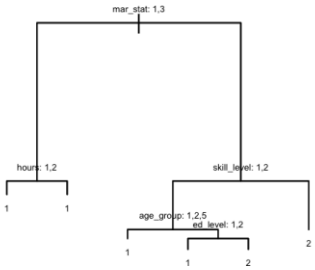


Figure 15: Trade-off between accuracy and complexity for tree models

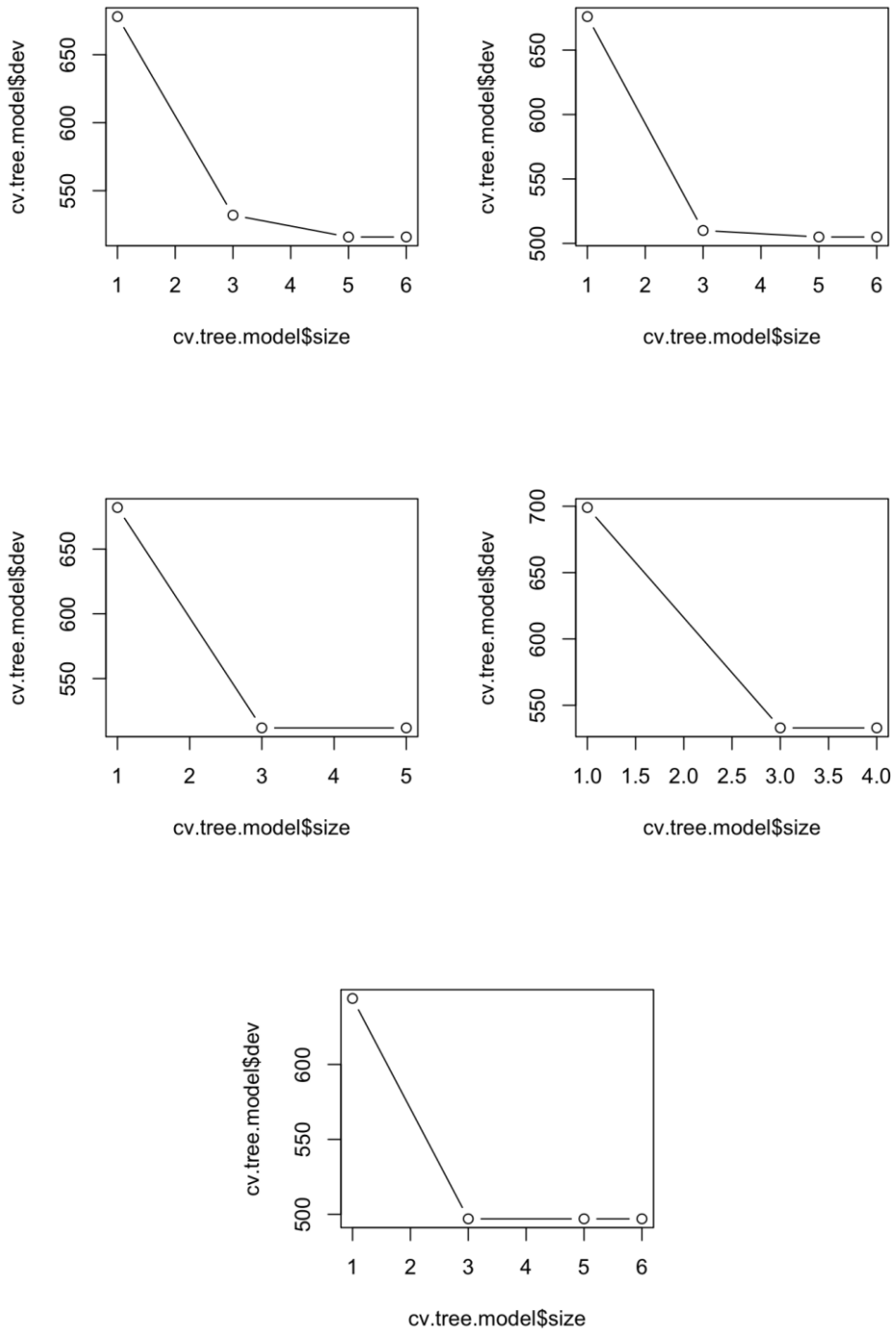
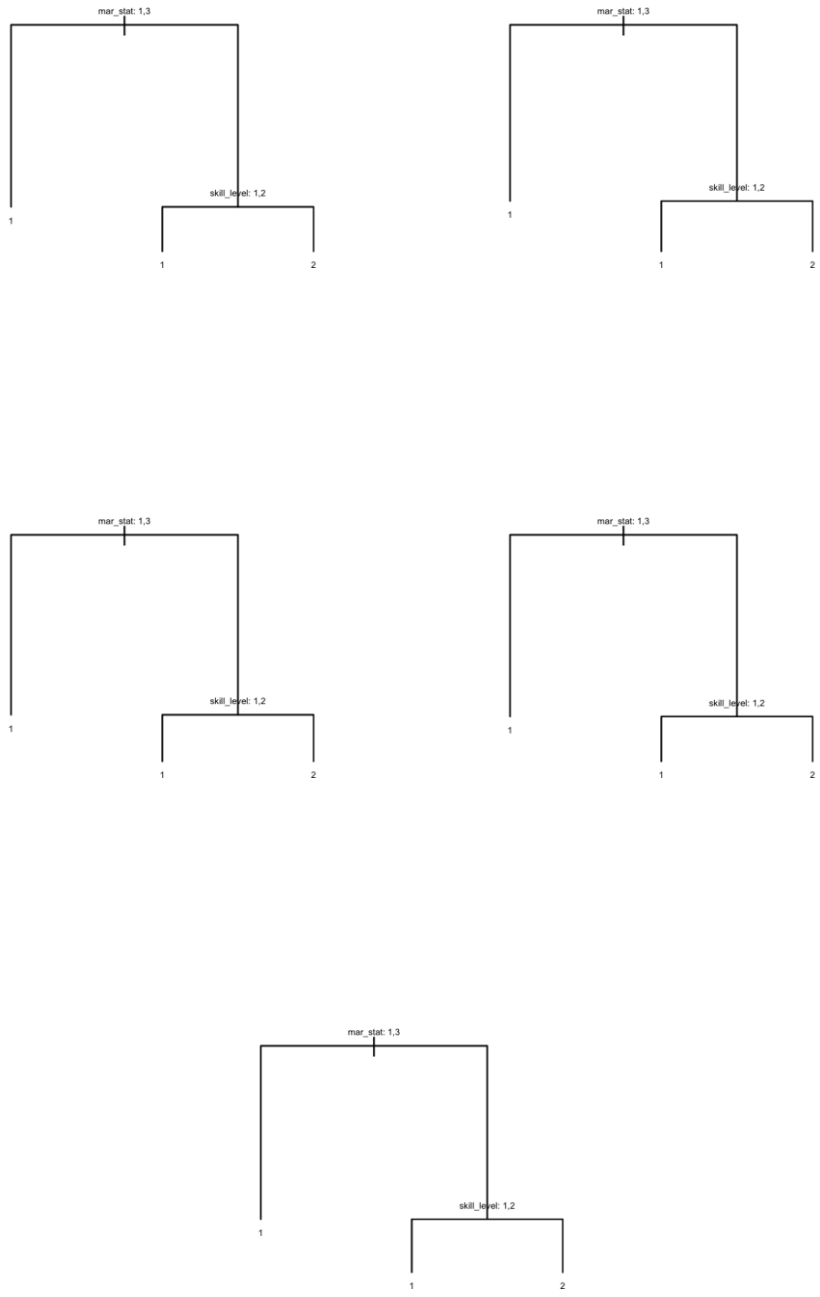


Figure 16: Pruned trees (3 leaves)



3.3. Ensemble methods

As classification trees failed to outperform logistic regression, a linear DB remains the most plausible interpretation. However, we cannot rule out a nonlinear DB without applying ensemble methods. Instead of using a single classification tree, bagging and RF aim to use many decorrelated trees to predict income level, which can produce more stable predictions. By comparing the predictive accuracy of these methods to standard classification techniques, we can make a more definitive conclusion about the nature of the DB. Moreover, bagging and RF are extremely useful for this analysis because they can be used to determine which variables are the best predictors of income level. This is vital to the selection of variables to include in the standard classification models and our interpretation later on.

First, we estimated a bagging model to determine whether classification trees failed to outperform logistic due to their high variance or the nature of the DB. This involved forming 500 random samples with replacement (bootstrapping), with each one being used to train a deep tree. This reduces the probability of samples being very similar, making the deep trees and their predictions more independent.

To help us understand which variables are the most important predictors of income level, we identified the observations which were left out of each sample (out of bag observations). These were then passed through the corresponding trees to produce predictions for income level, from which 'out of bag' (OOB) PFR could be calculated for each tree. The final 'OOB' PFR is given by the average across the 500 trees. Variable importance can then be inferred by repeating the exercise when certain explanatory variables are dropped. As indicated by *Figure 17*, dropping age group, education level, marital status, race and skill level were consistently the five variables which resulted in the largest increase in 'OOB' PFR. Therefore, bagging indicates these variables are important predictors of earnings.

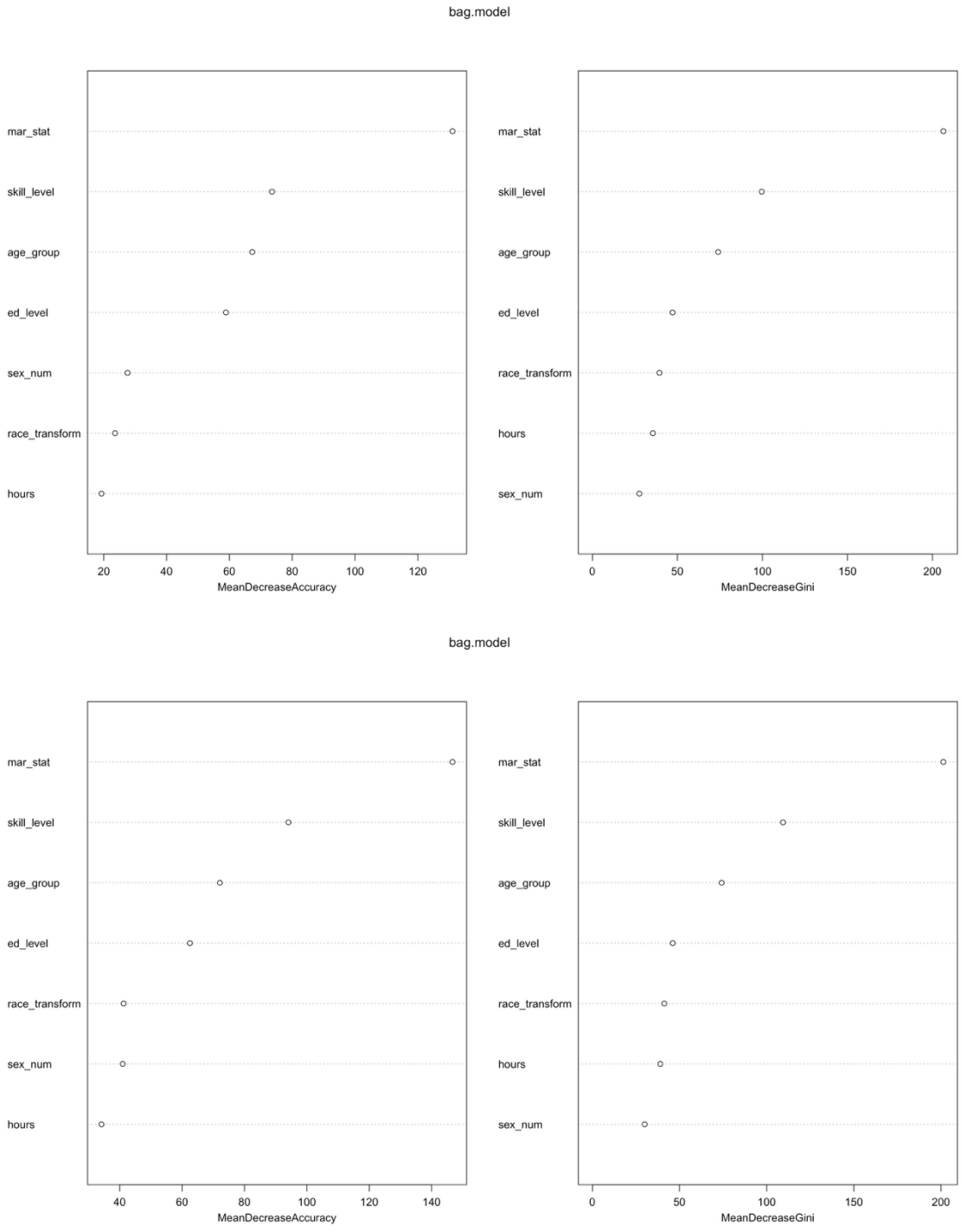
Next, we evaluated the predictive accuracy of bagging by passing each observation in the test sample through the 500 deep trees. This produced 500 predictions for income level per person, and the most common prediction was assigned, allowing us to calculate the test PFR. As seen in *Table 10*, the bagging models perform even worse than the pruned trees, let alone logistic. This calls into question whether the DB is non-linear. However, bagging models often struggle when there are highly important variables. This is because the deep trees aim to maximise the similarity of people in each region in terms of income (node purity). If certain

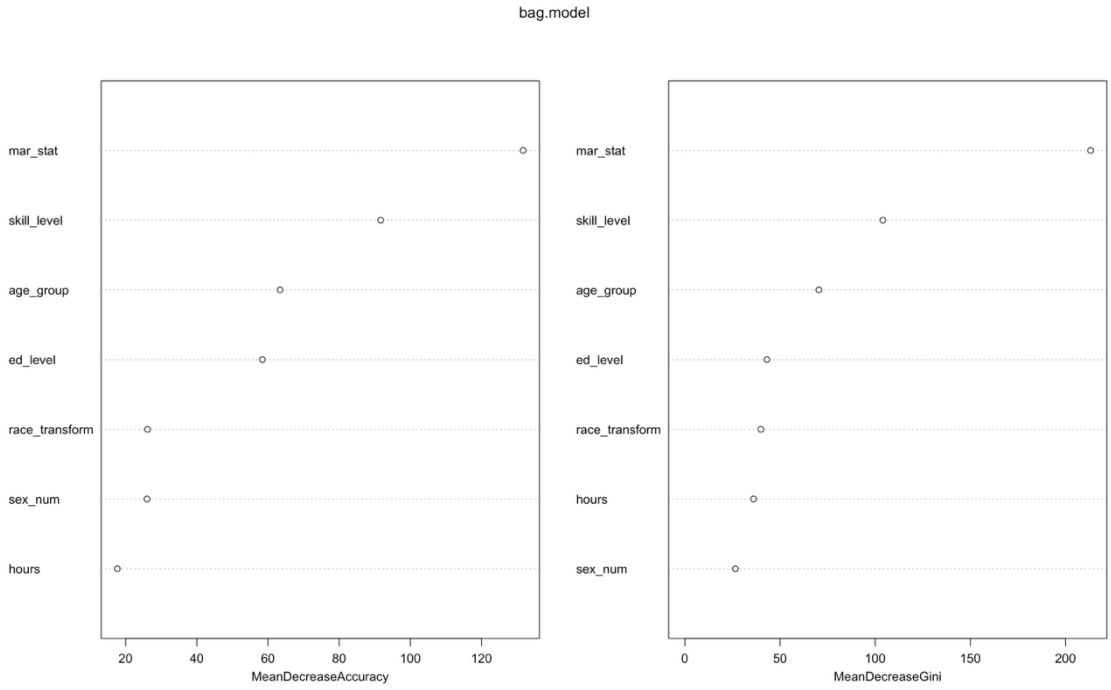
variables are particularly useful for this, other nuances can be overlooked. For instance, *Figure 17* indicates adding marital status to the bagging models results in a disproportionately large decrease in the Gini index, suggesting this variable is extremely useful for forming more similar groups in each region. However, earnings are likely related to many other factors. Therefore, to ensure we are not overlooking the possibility of a nonlinear DB, we next consider RF.

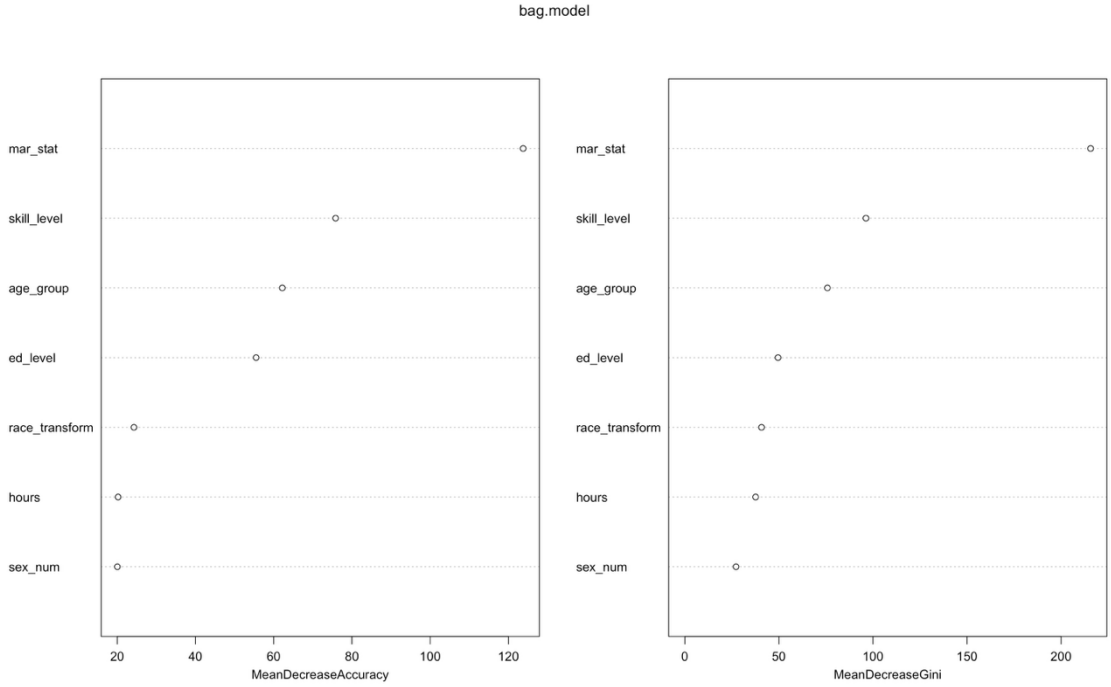
Table 10: PFR on test data for best performing classification techniques vs. deep trees vs. bagging models

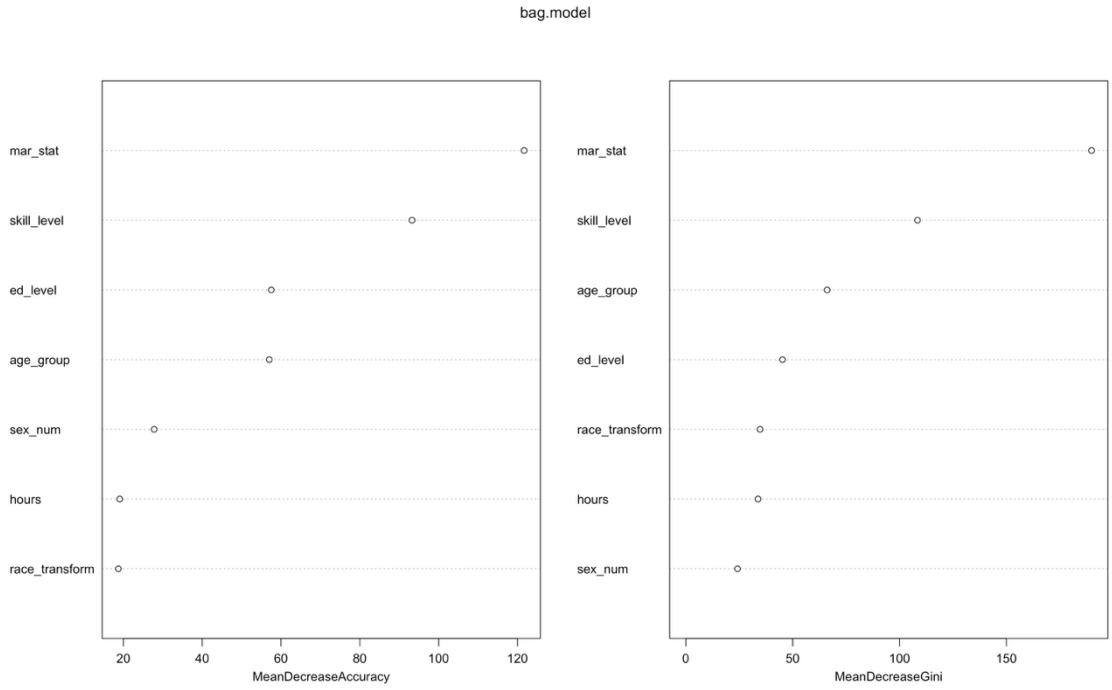
Model	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**	**Seed 5**
Logistic	0.173	0.180	0.175	0.166	0.193
Pruned tree	0.184	0.195	0.192	0.178	0.215
Bagging model	0.192	0.208	0.179	0.182	0.192

Figure 17: Variable importance implied by bagging




bag.model
mar_stat
skill_level
age_group
ed_level
race_transform
sex_num
hours
20
40
60
80
100
120
MeanDecreaseAccuracy
mar_stat
skill_level
age_group
ed_level
race_transform
hours
sex_num
0
50
100
150
200
MeanDecreaseGini


bag.model
mar_stat
skill_level
age_group
ed_level
race_transform
hours
sex_num
20
40
60
80
100
120
MeanDecreaseAccuracy
mar_stat
skill_level
age_group
ed_level
race_transform
hours
sex_num
0
50
100
150
200
MeanDecreaseGini



RF models are built in the same way as bagging models, except each tree is restricted to a random subset of variables. This is useful because not all trees will form splits using marital status and skill level, allowing potentially useful information from other variables to be extracted. Often, the size of the subset is set to $\sqrt{p}$, where p is the number of explanatory variables (Hastie, Tibshirani and Friedman, 2009). In our case, this is $\sqrt{7} \approx 2.65$, which rounds to three variables. Therefore, we estimated 500 trees using bootstrap sample selection, where each tree could only form splits using three randomly selected variables.

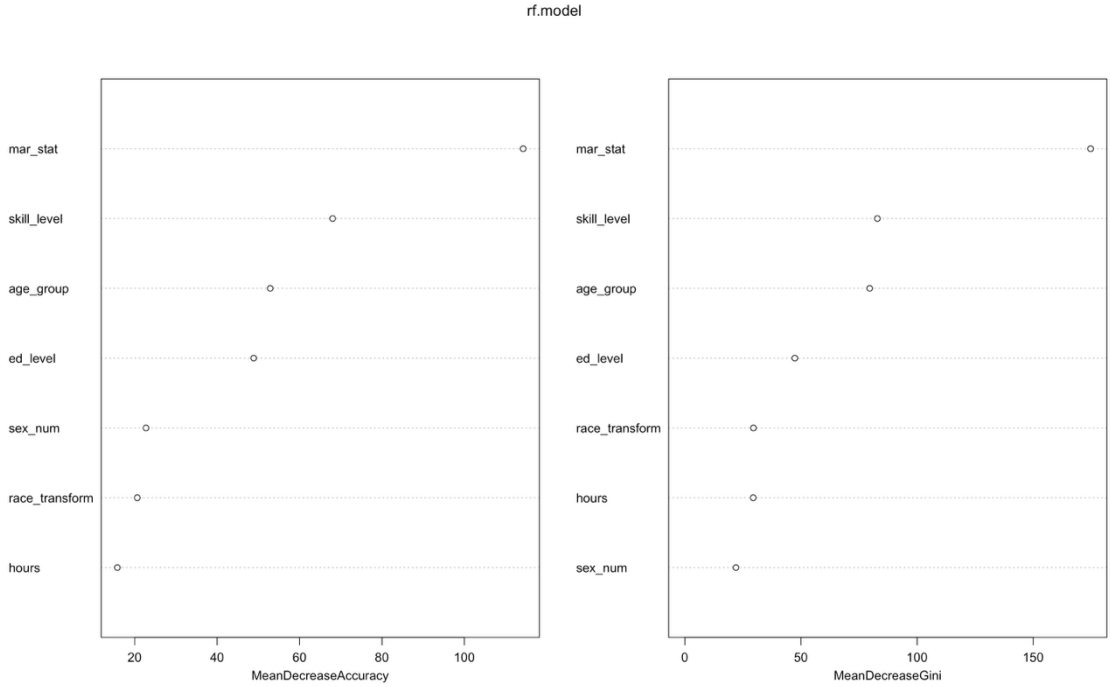
Using the same exercise as before, the 'OOB' observations were used to evaluate variable importance. *Figure 18* indicates the same five variables consistently displayed high importance. This allows us to confidently conclude these variables are the most important predictors of high income, particularly marital status and skill level. However, the advantage of RF is that information from other important variables, such as age group and skill level, will be more heavily utilised. This is demonstrated by the higher level of predictive accuracy for RF compared to bagging and classification trees, as shown by the comparison of test PFR in *Table 11*. Test PFR was similarly calculated after passing each observation in the test sample through the 500 trees and assigning the most common prediction to each person. However, the additional complexity of RF does not appear to offer superior predictive accuracy over logistic. This provides further evidence against there being a nonlinear DB for income level. Instead,

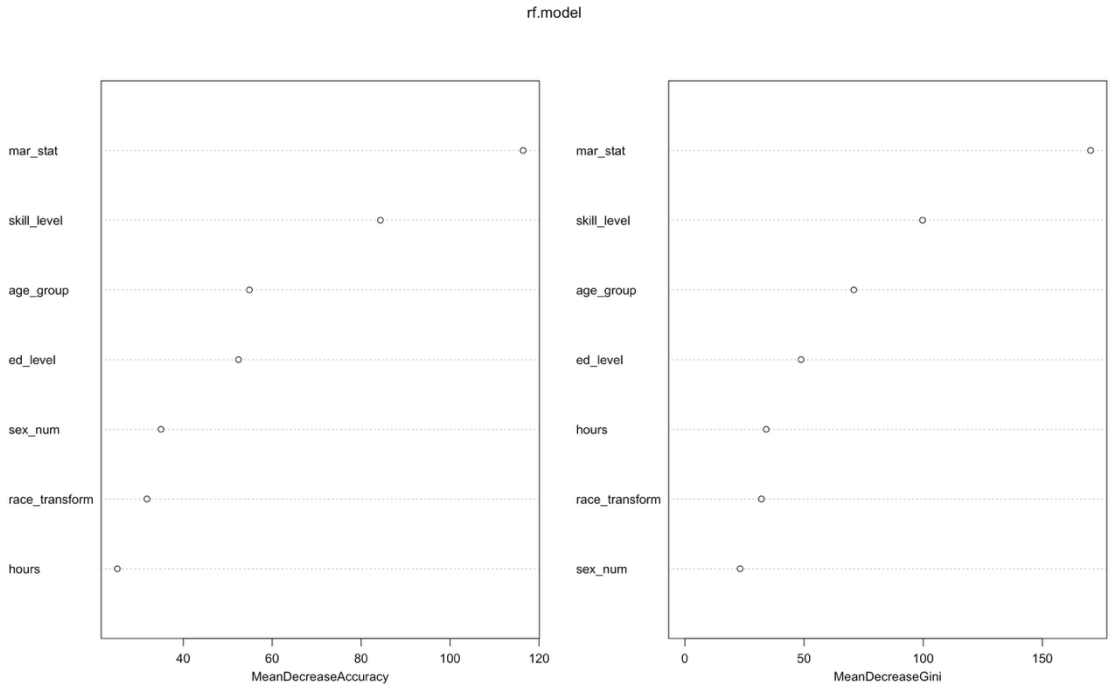
the linear DB of logistic appears more suitable. Nonetheless, this exercise was extremely helpful for identifying which variables are the most important predictors of income level.

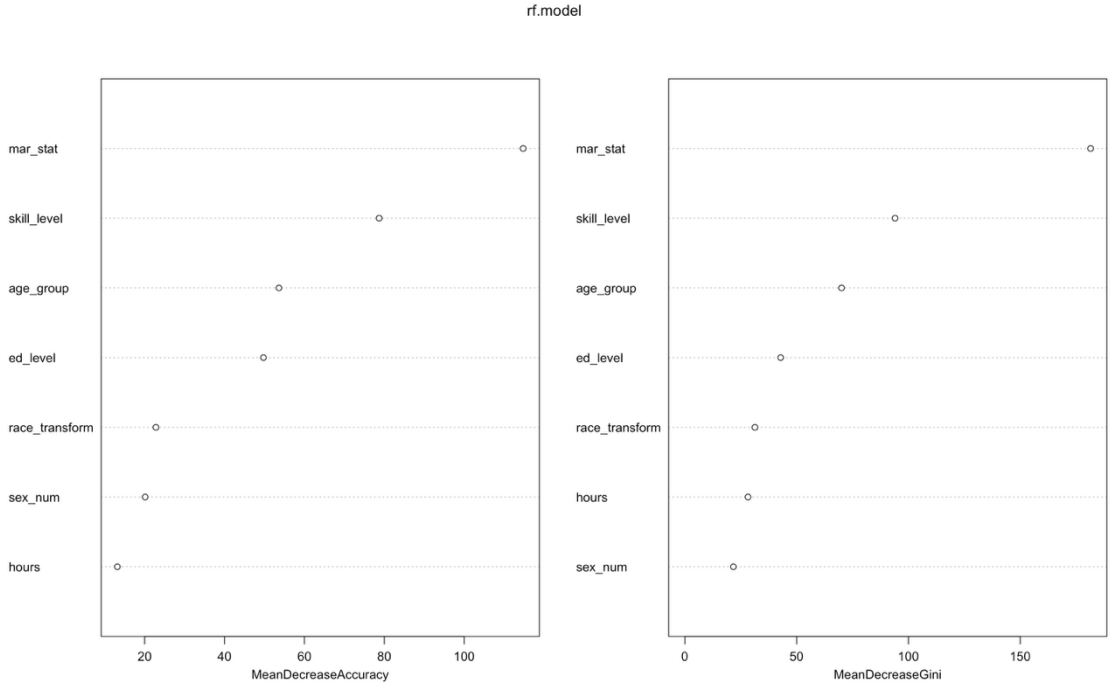
Table 11: PFR on test data for logistic vs. pruned trees vs. ensemble models

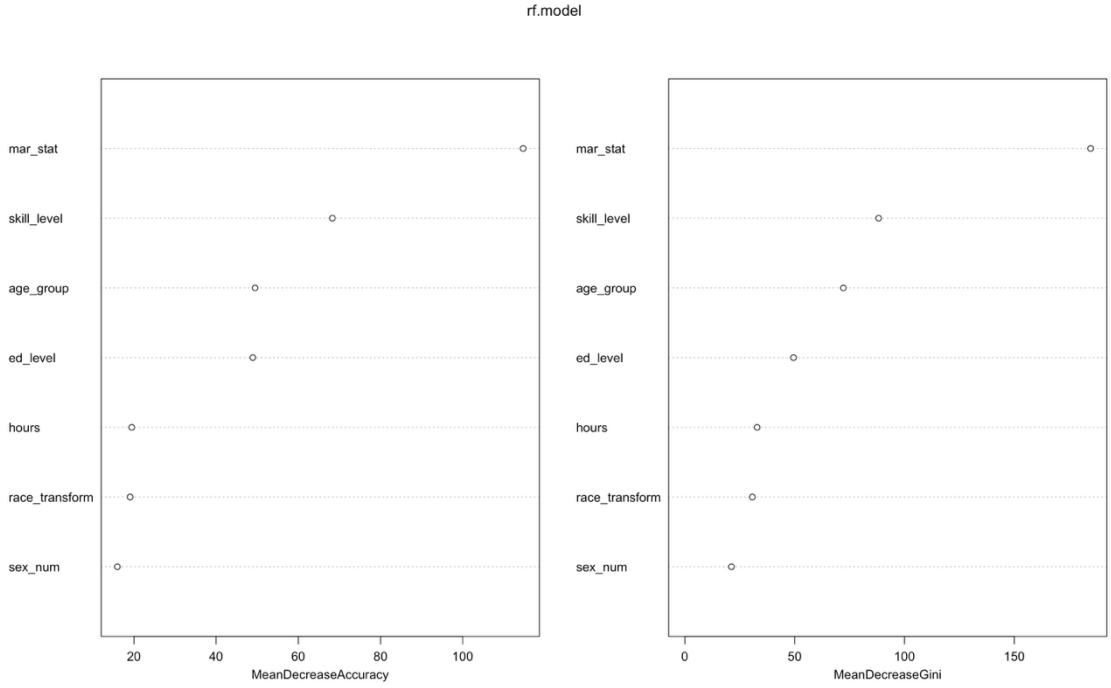
Model	Seed 1	Seed 2	Seed 3	Seed 4	Seed 5
Logistic	0.173	0.180	0.175	0.166	0.193
Pruned	0.184	0.195	0.192	0.178	0.215
Bagging model	0.192	0.208	0.179	0.182	0.192
Random forest	0.171	0.191	0.172	0.164	0.183

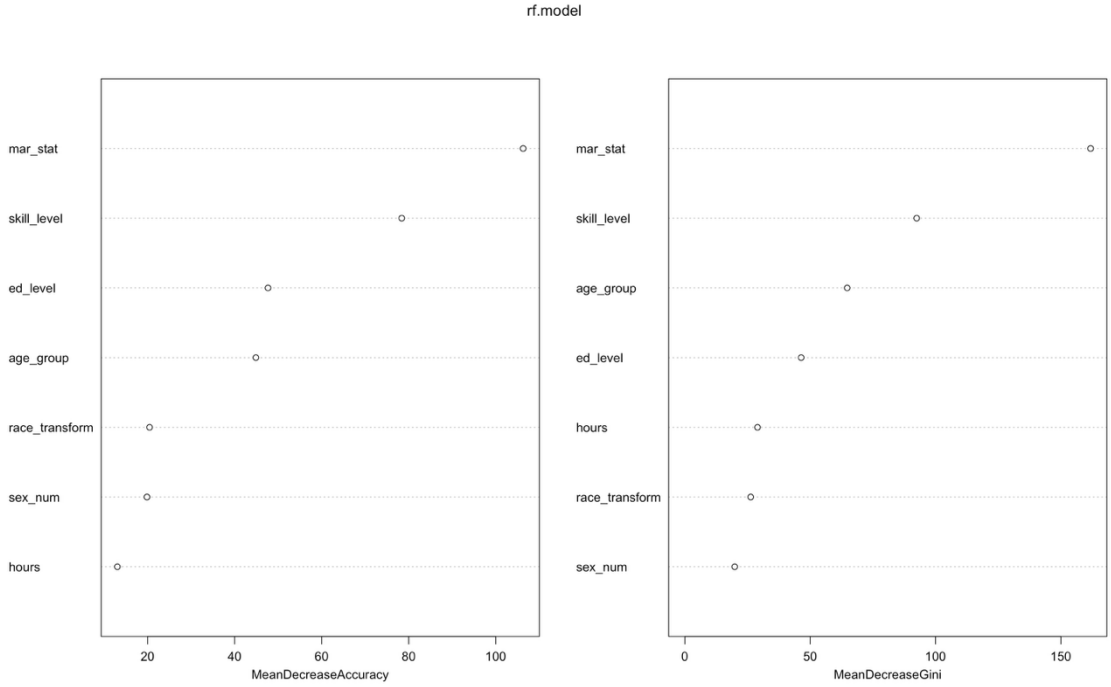
Figure 18: Variable importance implied by random forest




rf.model
mar_stat
skill_level
age_group
ed_level
sex_num
race_transform
hours
40
60
80
100
120
MeanDecreaseAccuracy
mar_stat
skill_level
age_group
ed_level
hours
race_transform
sex_num
0
50
100
150
MeanDecreaseGini


rf.model
mar_stat
skill_level
age_group
ed_level
race_transform
sex_num
hours
20
40
60
80
100
MeanDecreaseAccuracy
mar_stat
skill_level
age_group
ed_level
race_transform
hours
sex_num
0
50
100
150
MeanDecreaseGini


rf.model
mar_stat
skill_level
age_group
ed_level
hours
race_transform
sex_num
20
40
60
80
100
MeanDecreaseAccuracy
mar_stat
skill_level
age_group
ed_level
hours
race_transform
sex_num
0
50
100
150
MeanDecreaseGini


rf.model
mar_stat
skill_level
ed_level
age_group
race_transform
sex_num
hours
20
40
60
80
100
MeanDecreaseAccuracy
mar_stat
skill_level
age_group
ed_level
hours
race_transform
sex_num
0
50
100
150
MeanDecreaseGini

4. Robustness checks

The methodology and results uncovered the most important predictors of income level. When using these variables in a logistic regression, more complicated methods struggled to offer superior predictive accuracy. Therefore, a linear DB appears to be the most useful interpretation of the data. However, we may have overlooked a more complex relationship in favour of using the most interpretable method. Considering this, we also analyse income level using the most advanced methods available – GB and SNN. If they outperform logistic regression, this could suggest a more complex underlying relationship. We also repeat our analysis using a 50:50 validation set split to determine whether our results are a symptom of overfitting.

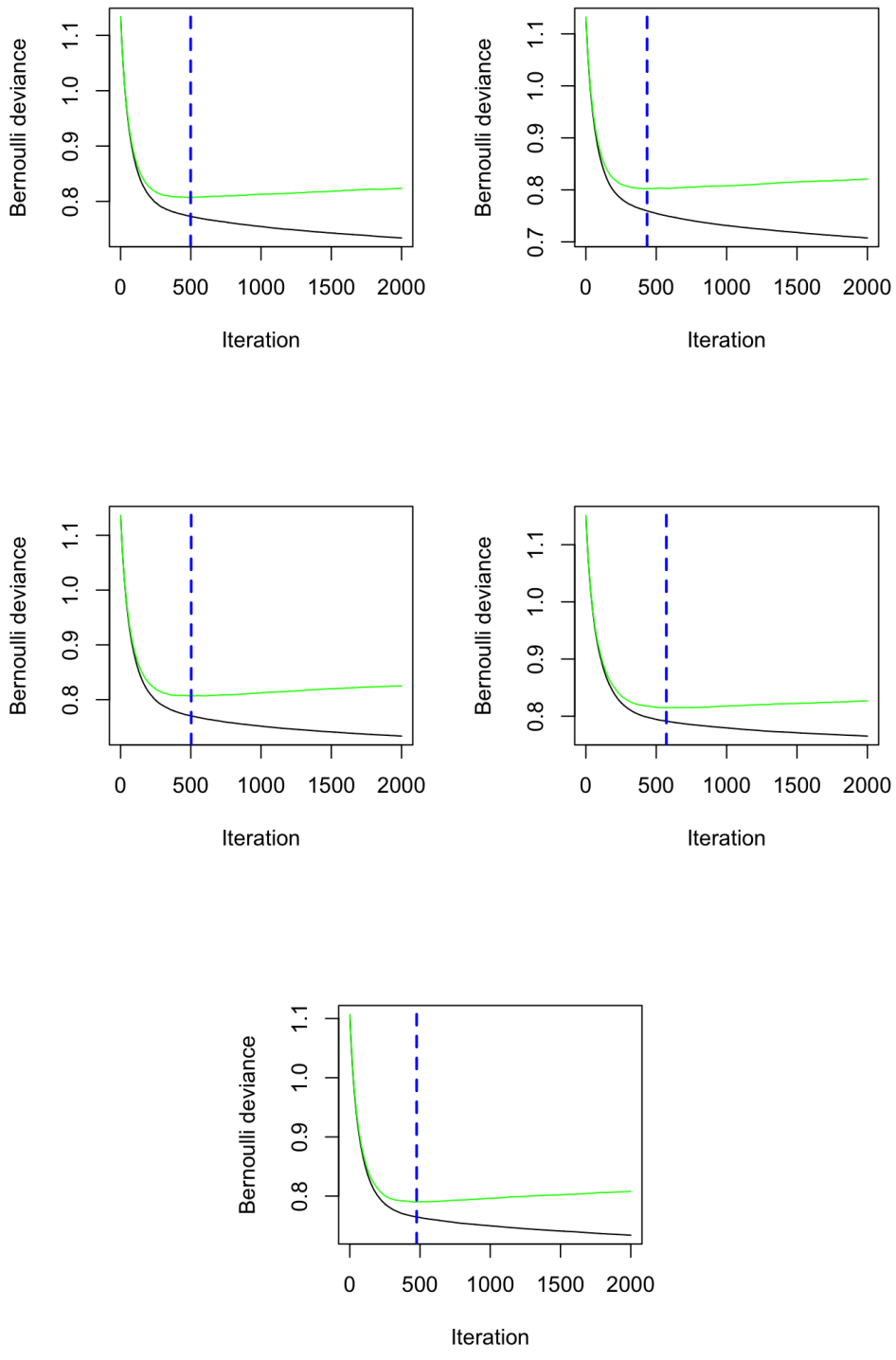
4.1. Alternative techniques

We first considered GB, which is also an ensemble method. This grows a series of trees, each designed to improve upon the last. We start by forming an initial prediction, then grow a tree to predict the error associated with this. After adjusting our initial prediction for the expected error, we grow another tree to predict the error associated with this new one. Our second prediction is then adjusted for its predicted error, and so on. We apply 5-fold cross-validation to the training data to determine the number of trees and interaction depth which maximise predictive accuracy. Based on the PFR for the test folds of the training data, the optimal number of trees is typically around 500, as indicated by the blue dashed lines in *Figure 19*. We also chose a low learning rate (0.01) to ensure the model is not overly sensitive to mistakes, which could result in overfitting. *Table 12* indicates GB offers a slightly higher level of predictive accuracy than logistic regression.

Table 12: PFR on test data for logistic vs. tree model vs. ensemble models

Model	Seed 1	Seed 2	Seed 3	Seed 4	Seed 5
Logistic	0.173	0.180	0.175	0.166	0.193
Prune tree	0.184	0.195	0.192	0.178	0.215
Bagging model	0.192	0.208	0.179	0.182	0.192
Random forest	0.171	0.191	0.172	0.164	0.183
Gradient boosting	0.168	0.190	0.173	0.163	0.184

Figure 19: Optimal number of trees for gradient boosting



Next, we consider SNN, which can capture complex nonlinear relationships and interactions between variables. More units in the hidden layer will allow the model to capture more complex patterns but risks overfitting. To balance this trade-off, we conducted 10-fold

cross-validation on the training data to choose the number of units which maximised predictive accuracy on the test folds of the training data, considering up to 10 units. The optimal number of units is reported in *Table 13*. We decided against cross-validation to tune the decay parameter, as tuning the number of units and decay would have been a significant computational burden. Instead, we chose a value of 0.001, which is small enough to allow the model to learn relationships, whilst penalising large weights, reducing the risk of overfitting.

Table 13: Optimal units for SNNs

Seed	Optimal Units
1	2
2	1
3	2
4	1
5	1

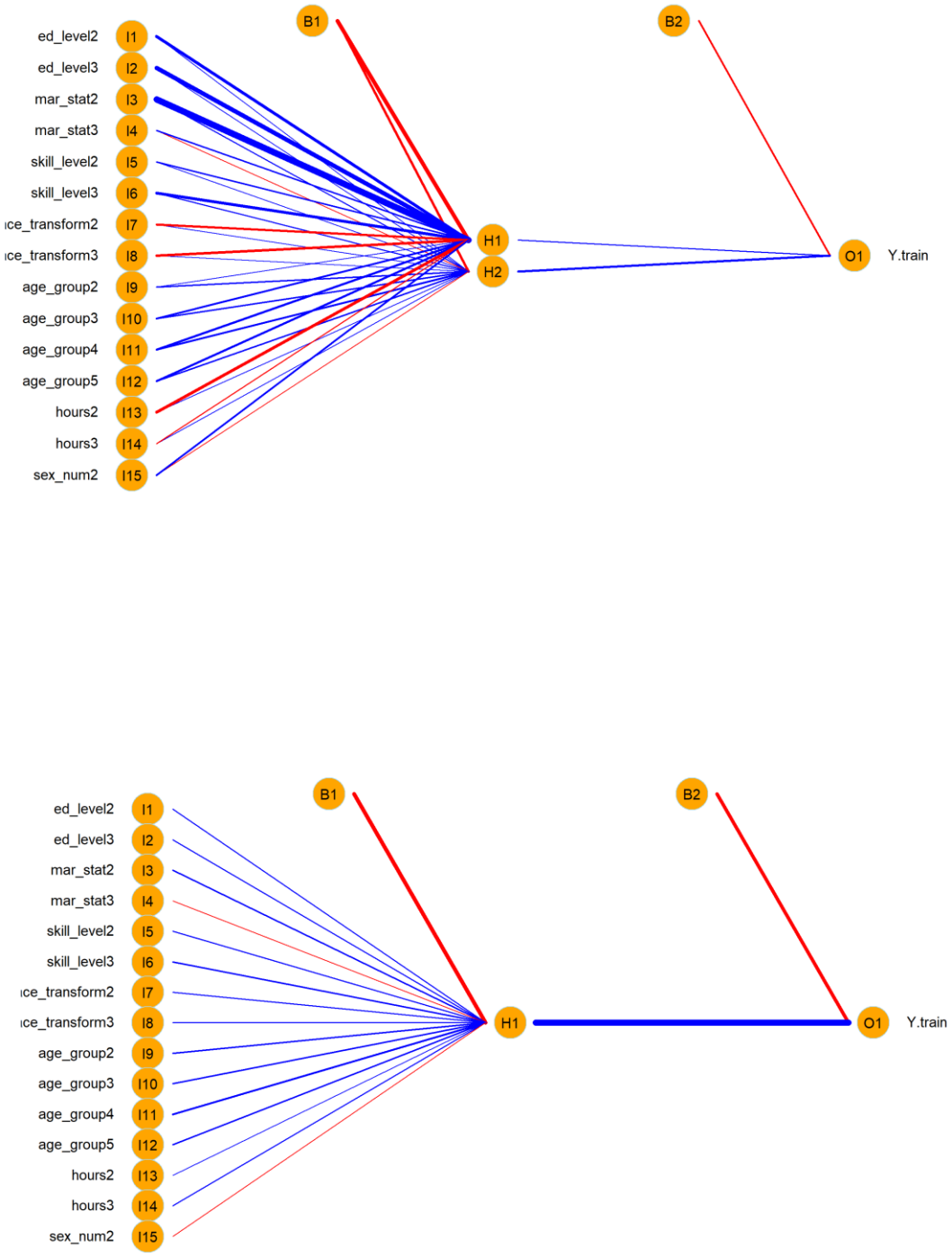
After choosing the hyperparameters, we estimated a shallow neural network on the training sample. The resulting networks are displayed in *Figure 20*. We then evaluated the model's performance on training and test data. *Table 14* indicates both GB and SNN struggled to outperform logistic.

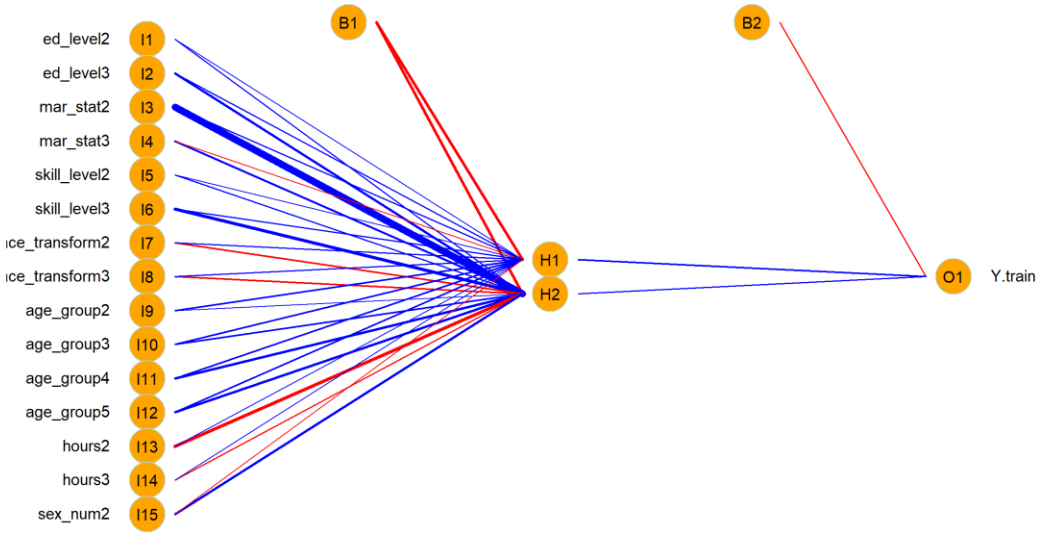
Table 14: PFR on test data for main models

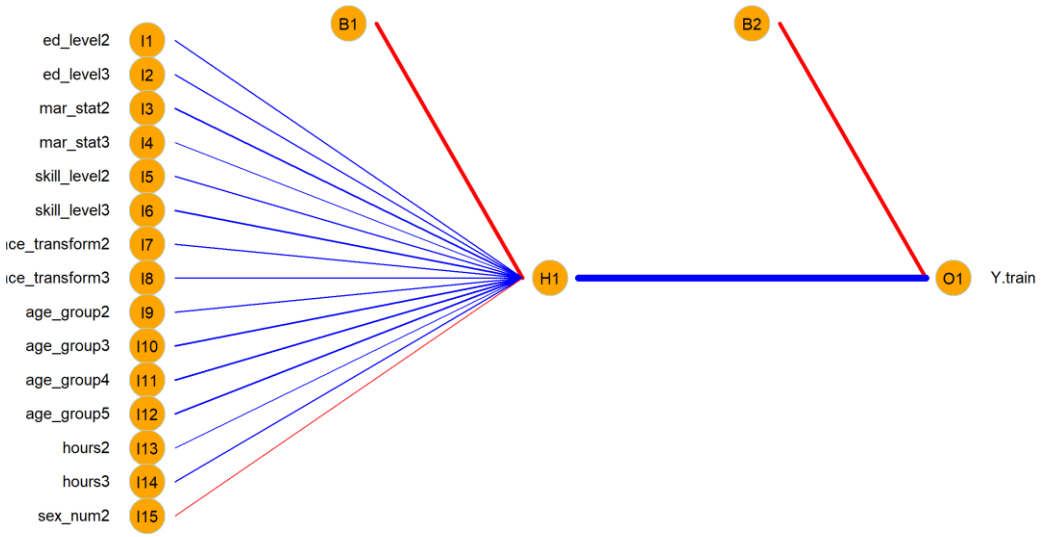
Model	Seed 1	Seed 2	Seed 3	Seed 4	Seed 5
Logistic	0.173	0.180	0.175	0.166	0.193
Prune tree	0.184	0.195	0.192	0.178	0.215
Bagging model	0.192	0.208	0.179	0.182	0.192
Random forest	0.171	0.191	0.172	0.164	0.183
Gradient boosting	0.168	0.190	0.173	0.163	0.184
Shallow NN	0.166	0.190	0.175	0.167	0.191

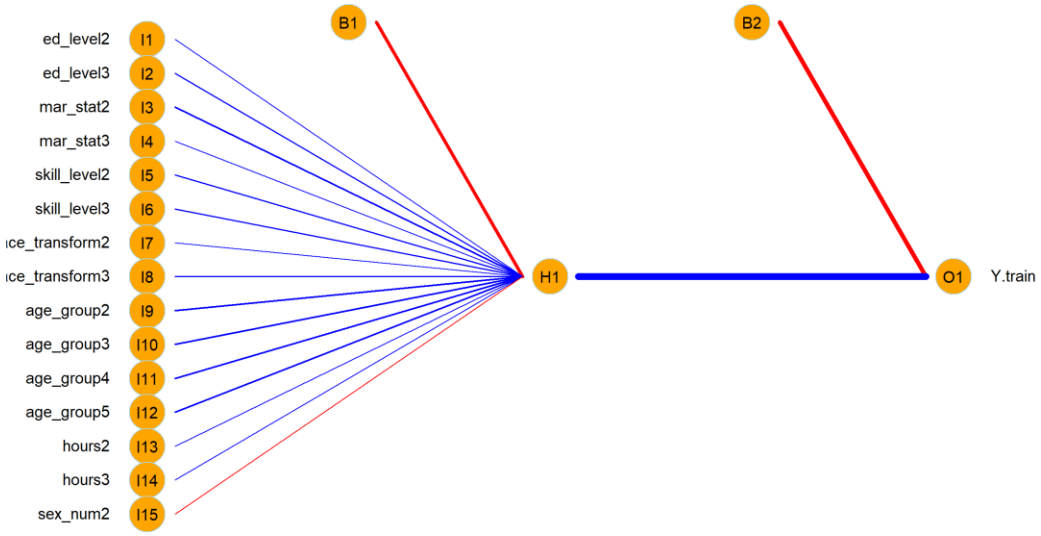
Note: PFR shown to 3 d.p.

Figure 20: Shallow Neural Networks




ed_level2
ed_level3
mar_stat2
mar_stat3
skill_level2
skill_level3
ice_transform2
ice_transform3
age_group2
age_group3
age_group4
age_group5
hours2
hours3
sex_num2
I1
I2
I3
I4
I5
I6
I7
I8
I9
I10
I11
I12
I13
I14
I15
B1
B2
H1
H2
O1
Y.train


ed_level2
ed_level3
mar_stat2
mar_stat3
skill_level2
skill_level3
ice_transform2
ice_transform3
age_group2
age_group3
age_group4
age_group5
hours2
hours3
sex_num2
I1
I2
I3
I4
I5
I6
I7
I8
I9
I10
I11
I12
I13
I14
I15
B1
B2
H1
O1
Y.train



4.2. Alternative validation set split

Finally, we evaluate the robustness of our results using a 50:50 validation set split to determine whether our initial results are a symptom of overfitting. *Table 15* indicates logistic regression demonstrated the highest level of predictive accuracy for this split, which could suggest logistic is the most robust and suitable interpretation of this data. The stronger performance of more complex models on the 70:30 split could have been due to overfitting. However, maybe the dataset is too small for complex relationships to be uncovered because more advanced models require more training data.

Table 15: PFR on test data for main models with 50:50 validation set split

Model	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**	**Seed 5**
Logistic	0.176	0.193	0.176	0.172	0.186
Pruned tree	0.194	0.204	0.197	0.185	0.206
Bagging model	0.194	0.202	0.191	0.193	0.201
Random forest	0.184	0.192	0.179	0.179	0.192
Gradient boosting	0.182	0.197	0.181	0.176	0.189
Shallow NN	0.182	0.195	0.183	0.176	0.188

5. Discussion

This section interprets the results and their robustness, as well as discussing their implications. We also aim to highlight the strengths and weaknesses of our approach to ensure our conclusions are formed with reasonable considerations in mind.

5.1. Interpretation of results

We have used various techniques to analyse income, aiming to determine the nature of the DB and most important predictors of income level. Whilst some more advanced methods slightly outperformed logistic, this was marginal and not robust across the alternative validation set split. This suggests the DB is relatively smooth. Instead of changing abruptly for people with certain characteristics, factors like marital status, skill level and age probably contribute some upward or downward pressure to the probability of being a high earner. Additionally, the variable importance plots from bagging and RF (*Figures 16 and 17*) produced consistent results, identifying marital status, skill level, education and age group as having high predictive accuracy. Therefore, we use these variables to estimate a logistic regression using the entire dataset and interpret our data. This model is presented in *Table 16*.

Table 16: Logistic model

Variable	**Meaning**	**Coefficient (Log odd)**
(Intercept)	'Baseline' individual	-6.030
Mar_stat2	Married	2.328
Mar_stat3	Never married	0.320
Skill_level2	Medium skill	0.609
Skill_level3	High skill	1.602
Age_group2	Age 25-34	0.518
Age_group3	Age 35-49	1.253
Age_group4	Age 50-65	1.404
Age_group5	Older than 65	1.017
Ed_level2	High School	0.611
Ed_level3	Graduate	1.199
Race_transform2	Black	0.574
Race_transform3	White	0.775

The intercept shows the log odds of a 'baseline' individual having high income. This person is no longer married, younger than 25, has no high school education and belonged to the 'other'

race category. Such an individual is estimated to have log odds of –6.030 of high income. This translates into a very low probability of 0.24%. The variable with the largest impact on the probability of having high income is being married, with a coefficient of 2.328. Therefore, if the 'baseline' individual was otherwise identical except they are married, their log odds of high income become –3.7022, which translates into a probability of 2.38%. This suggests there is a 'marriage premium', where married people are much more likely to be high earners. Perhaps married people pursue higher wages due to responsibilities often associated with marriage, such as raising children. Or perhaps it could be that high earners are more likely to get married. Importantly, we are not determining causality. We can also see that having a high skilled job and being a graduate both have positive impacts on high income. We noticed that the probability of having high income generally increases with age, except for the oldest group, and that white individuals are assigned a higher probability than other ethnic groups. Overall, this model predicts that the individual with the highest chance of having high income is married, highly skilled, a graduate, between 50 and 65 and white with log odds of 1.278, and thus a probability of 78.2%. This demonstrates a huge contrast between the characteristics of those most and least likely to have high income.

5.2. Strengths and weaknesses of this analysis

When evaluating and considering our analysis of the data in part 2, certain strengths and weaknesses of our approach became apparent. The main strengths are evaluating across seeds 1-5, using a 50:50 as a robustness check and using a broad range of methods. The main weaknesses are the dataset may have been too small for more complex methods to realise their potential, and we did not use cross-validation to fully optimise all models.

Looking at the first strength, evaluating results across five seeds allows us to check whether the results were driven by a particular training set. Whilst individual PFRs may differ across seeds, the models' rankings generally stayed consistent. This gave us more confidence in concluding the structure of the DB and the predictors of high income.

Furthermore, when running all the models across seeds 1-5 with a 50:50 and 70:30 split it allows us to check whether the results from the 70:30 split were overfitting or consistent. When evaluating models across both splits, logistic showed as the overall strongest performer, further reinforcing the conclusion that a linear DB is the most appropriate interpretation of the data.

Additionally, using a wide range of methods, from logistic to SNN, meant we considered many interpretations of the data. By comparing the performance of these methods, we believe we gained stronger insights into the nature of earnings. However, only having 4,000 observations might mean some models having insufficient training data. This means we cannot conclude logistic would outperform other methods on larger datasets.

Finally, another weakness is not using cross-validation to select certain hyperparameters. For instance, in the GB model we set the learning rate to 0.01, and in the SNN model we set the decay parameter to 0.001. Therefore, we may not have used these models to their full potential. However, this would have carried a significant computational burden.

6. Conclusion

The objective of this analysis was to analyse income using a range of personal characteristics. We aimed to determine both important predictors of earnings and the best way to interpret the underlying DB. Through the application of various methods, ranging from standard classification techniques to SNN, we gained some key insights. Firstly, marital status, skill level, education level and age group were the most significant predictors of income level. Secondly, whilst complex 'black box' models were applied, they only offered a marginal improvement in predictive accuracy. The strong performance of logistic regression suggests there might be a more linear DB for income level in terms of these variables. The robustness checks supported this conclusion because logistic remained the strongest performer across an alternative validation set split. Furthermore, our results demonstrate how simple models can often perform just as well as complex methods, whilst offering us more economic interpretation of the underlying data.

7. References

Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edn. New York: Springer. Available at: Springer PDF [Accessed: 15 April 2026].